COURS FBD

RAPPEL

1- MCD

Soit la base de données du projet GESETUDE qui consiste en l'informatisation du système de gestion d'une Université dont voici les tables :

- Professeur
- Matière
- Etudiant
- Filière
- Classe

Règles de gestion :

- Un professeur peut enseigner plusieurs matières et une matière peut être enseigner par plusieurs professeurs
- Dans une filière, on enseigne plusieurs matières et une matière peut être enseigner dans plusieurs filières
- Un élève ne peut être inscrit que dans une filière et dans une filière sont inscrit plusieurs élèves
- Une matière est programmée dans une classe pour un professeur a une date donnée.

PRESENTER LE MCD DU PROJET GESETUDE.

2- MLD

A PARTIR DU MCD, PRESENTEZ LE MLD DU PROJET GESETUDE.

CHAPITRE I – INTRODUCTION AUX BASES DE DONNEES

1. NOTION DE BASE DE DONNEES

a. Description Générale

Il est difficile de donner une définition exacte de la notion de base de données. Une définition très générale pourrait être :

Définition 1 -Base de données- Un ensemble organisé d'informations avec un objectif commun.

Peu importe le support utilisé pour rassembler et stocker les données (papier, fichiers, etc.), dès lors que des données sont rassemblées et stockées d'une manière organisée dans un but spécifique, on parle de base de données.

Plus précisément, on appelle base de données un ensemble structuré et organisé permettant le stockage de grandes quantités d'informations afin d'en faciliter l'exploitation (ajout, mise à jour, recherche de données).

b. Base de données informatisée

Définition 2 -Base de données informatisée- Une base de données informatisée est un ensemble structuré de données enregistrées sur des supports accessibles par l'ordinateur, représentant des informations du monde réel et pouvant être interrogées et mises à jour par une communauté d'utilisateurs.

Le résultat de la conception d'une base de données informatisée est une description des données. Par description on entend définir les propriétés d'ensembles d'objets modélisés dans la base de données et non pas d'objets particuliers.

Cette description des données est réalisée en utilisant un modèle de données. Ce dernier est un outil formel utilisé pour comprendre l'organisation logique des données.

2. MODELE DE BASE DE DONNEES

a. Modèle hiérarchique

Une base de données hiérarchique est une forme de système de gestion de base de données qui lie des enregistrements dans une structure arborescente de façon à ce que chaque enregistrement n'ait qu'un seul possesseur (par exemple, une paire de chaussures n'appartient qu'à une seule personne).

b. Modèle réseau

Le modèle réseau est en mesure de lever de nombreuses difficultés du modèle hiérarchique grâce à la possibilité d'établir des liaisons de type n-n, les liens entre objets pouvant exister sans restriction. Pour retrouver une donnée dans une telle modélisation, il faut connaître le chemin d'accès (les liens) ce qui rend les programmes dépendants de la structure de données.

c. Modèle relationnel

Une base de données relationnelle est une base de données structurée suivant les principes de l'algèbre relationnelle.

Ce modèle propose de stocker des données hétérogènes dans des tables, permettant d'établir des relations entre elles. De nos jours, ce modèle est extrêmement répandu, mais en 1970, cette idée était considérée comme une curiosité intellectuelle. On doutait que les tables puissent être jamais gérées de manière efficace par un ordinateur.

d. Modèle objet

La notion de bases de données objet ou relationnel-objet est plus récente et encore en phase de recherche et de développement. Elle sera très probablement ajoutée au modèle relationnel.

3. SYSTEME DE GESTION DE BASE DE DONNEES (SGBD)

Ensemble de logiciels systèmes permettant aux utilisateurs d'insérer, de modifier, et de rechercher efficacement des données spécifiques dans une grande masse d'informations (pouvant atteindre plusieurs milliards d'octets) partagée par de multiples utilisateurs. Un système de gestion de bases de données héberge généralement plusieurs bases de données, qui sont destinées à des logiciels ou des thématiques différents.

Actuellement, la plupart des SGBD fonctionnent selon un mode client/serveur. Le serveur (sous-entendu la machine qui stocke les données) reçoit des requêtes de plusieurs clients et ceci de manière concurrente.

a. Objectifs

Des objectifs principaux ont été fixés aux SGBD dès l'origine de ceux-ci, et ce, afin de résoudre les problèmes causés par la démarche classique. Ces objectifs sont les suivants :

Indépendance physique :

- La façon dont les données sont définies doit être indépendante des structures de stockage utilisées.

Indépendance logique :

- Un même ensemble de données peut être vu différemment par des utilisateurs différents. Toutes ces visions personnelles des données doivent être intégrées dans une vision globale.

Accès aux données :

- L'accès aux données se fait par l'intermédiaire d'un Langage de Manipulation de Données (LMD). Il est crucial que ce langage permette d'obtenir des réponses aux requêtes en un temps « raisonnable ». Le LMD doit donc être optimisé, minimiser le nombre d'accès disques, et tout cela de façon totalement transparente pour l'utilisateur.

Administration centralisée des données (intégration) :

- Toutes les données doivent être centralisées dans un réservoir unique commun à toutes les applications. En effet, des visions différentes des données (entre autres) se résolvent plus facilement si les données sont administrées de façon centralisée.

Non-redondance des données :

- Afin d'éviter les problèmes lors des mises à jour, chaque donnée ne doit être présente qu'une seule fois dans la base.

Cohérence des données :

- Les données sont soumises à un certain nombre de contraintes d'intégrité qui définissent un état cohérent de la base. Elles doivent pouvoir être exprimées simplement et vérifiées automatiquement à chaque insertion, modification ou suppression des données. Les contraintes d'intégrité sont décrites dans le Langage de Description de Données (LDD).

Partage des données :

- Il s'agit de permettre à plusieurs utilisateurs d'accéder aux mêmes données au même moment de manière transparente. Si ce problème est simple à résoudre quand il s'agit uniquement d'interrogations, cela ne l'est plus quand il s'agit de modifications dans un contexte multiutilisateur, car il faut : permettre à deux (ou plus) utilisateurs de modifier la même donnée « en même temps » et assurer un résultat d'interrogation cohérent pour un utilisateur consultant une table pendant qu'un autre la modifie.

Sécurité des données :

- Les données doivent pouvoir être protégées contre les accès non autorisés. Pour cela, il faut pouvoir associer à chaque utilisateur des droits d'accès aux données.

Résistance aux pannes :

- Que se passe-t-il si une panne survient au milieu d'une modification, si certains fichiers contenant les données deviennent illisibles ? Il faut pouvoir récupérer une base dans un état « sain ». Ainsi, après une panne intervenant au milieu d'une modification deux solutions sont possibles : soit récupérer les données dans l'état

dans lequel elles étaient avant la modification, soit terminer l'opération interrompue.

4. QUELQUES SGBD CONNUS ET UTILISES

Il existe de nombreux systèmes de gestion de bases de données, en voici une liste non exhaustive :

PostgreSQL, MySQL , Oracle , IBM DB2 , Microsoft SQL , Sybase, Informix.

CHAPITRE II – BASES DE DONNEES RELATIONNELLES

1. INTRODUCTION AU MODELE RELATIONNEL

Dans ce modèle, les données sont représentées par des tables. Les tables constituent donc la structure logique du modèle relationnel. Au niveau physique, le système est libre d'utiliser n'importe quelle technique de stockage (fichiers séquentiels, indexage, adressage dispersé, séries de pointeurs, compression…) dès lors qu'il est possible de relier ces structures à des tables au niveau logique.

Les objectifs du modèle relationnel sont :

- proposer des schémas de données faciles à utiliser ;
- améliorer l'indépendance logique et physique (cf. section 1.2.2) ;
- mettre à la disposition des utilisateurs des langages de haut niveau ;
- optimiser les accès à la base de données ;
- améliorer l'intégrité et la confidentialité ;
- fournir une approche méthodologique dans la construction des schémas.

De façon informelle, on peut définir le modèle relationnel de la manière suivante :

- les données sont organisées sous forme de tables à deux dimensions, encore appelées relations, dont les lignes sont appelées n-uplet ou tuple en anglais ;
- les données sont manipulées par des opérateurs de l'algèbre relationnelle ;
- l'état cohérent de la base est défini par un ensemble de contraintes d'intégrité.

2. ELEMENTS DU MODELE RELATIONNEL

Définition 1 -attribut- Un attribut est un identificateur (un nom) décrivant une information stockée dans une base.

Exemples d'attribut : l'âge d'une personne, le nom d'une personne, le numéro de sécurité sociale.

Définition 2 -Domaine- Le domaine d'un attribut est l'ensemble, fini ou infini, de ses valeurs possibles.

Par exemple, l'attribut numéro de sécurité sociale a pour domaine l'ensemble des combinaisons de quinze chiffres et nom a pour domaine l'ensemble des combinaisons de lettres (une combinaison comme cette dernière est généralement appelée chaîne de caractères ou, plus simplement, chaîne).

Définition 3 -relation- Une relation est un sous-ensemble du produit cartésien de n domaines d'attributs ($n > 0$).

Une relation est représentée sous la forme d'un tableau à deux dimensions dans lequel les n attributs correspondent aux titres des n colonnes.

Définition 4 -degré- Le degré d'une relation est son nombre d'attributs.

Définition 5 -occurrence ou n-uplets ou tuples- Une occurrence, ou n-uplets, ou tuples, est un élément de l'ensemble figuré par une relation. Autrement dit, une occurrence est une ligne du tableau qui représente la relation.

Définition 6 -cardinalité- La cardinalité d'une relation est son nombre d'occurrences.

Définition 7 -clé candidate- Une clé candidate d'une relation est un ensemble minimal des attributs de la relation dont les valeurs identifient à coup sûr une occurrence.

La valeur d'une clé candidate est donc distincte pour tous les tuples de la relation. La notion de clé candidate est essentielle dans le modèle relationnel.

Règle 8 Toute relation a au moins une clé candidate et peut en avoir plusieurs.

Définition 11 -clé étrangère- Une clé étrangère dans une relation est formée d'un ou plusieurs attributs qui constituent une clé primaire dans une autre relation.

Définition 12 -schéma relationnel- Un schéma relationnel est constitué par l'ensemble des schémas de relation.

Définition 13 -base de données relationnelle- Une base de données relationnelle est constituée par l'ensemble des n-uplets des différentes relations du schéma relationnel.

3. PASSAGE DU MODELE ENTITES-ASSOCIATIONS AU MODELE RELATIONNEL

Confert Cours de MERISE

4. ALGEBRE RELATIONNELLE

a. Introduction

L'algèbre relationnelle est un support mathématique cohérent sur lequel repose le modèle relationnel. L'objet de cette section est d'aborder l'algèbre relationnelle dans le but de décrire les opérations qu'il est possible d'appliquer sur des relations pour produire de nouvelles relations. L'approche suivie est donc plus opérationnelle que mathématique.

On peut distinguer trois familles d'opérateurs relationnels :

Les opérateurs unaires (Sélection, Projection) :

- ce sont les opérateurs les plus simples, ils permettent de produire une nouvelle table à partir d'une autre table.

Les opérateurs binaires ensemblistes (Union, Intersection Différence) :

- ces opérateurs permettent de produire une nouvelle relation à partir de deux relations de même degré et de même domaine.

Les opérateurs binaires ou n-aires (Produit cartésien, Jointure, Division) :

- ils permettent de produire une nouvelle table à partir de deux ou plusieurs autres tables.

Les notations ne sont pas standardisées en algèbre relationnelle. Ce cours utilise des notations courantes, mais donc pas forcément universelles.

b. Sélection

Définition 26 -sélection- La sélection (parfois appelée restriction) génère une relation regroupant exclusivement toutes les occurrences de la relation R qui satisfont l'expression logique E, on la note σ(E)R.

Il s'agit d'une opération unaire essentielle dont la signature est :

relation × expression logique → relation

En d'autres termes, la sélection permet de choisir (*i.e.* sélectionner) des lignes dans le tableau. Le résultat de la sélection est donc une nouvelle relation qui a les mêmes attributs que *R*. Si *R* est vide (*i.e.* ne contient aucune occurrence), la relation qui résulte de la sélection est vide.

Le tableau 3.6 montre un exemple de sélection.

Numéro	Nom	Prénom
5	**Durand**	**Caroline**
1	**Germain**	**Stan**
12	**Dupont**	**Lisa**
3	**Germain**	**Rose-Marie**

Tableau 3.5 : Exemple de relation Personne

Numéro	Nom	Prénom
5	Durand	Caroline
12	Dupont	Lisa

Tableau 3.6 : Exemple de sélection sur la relation Personne du tableau 3.5 : σ(Numéro≥5)Personne

c. Projection

Définition 27 -projection- La projection consiste à supprimer les attributs autres que A_1,… A_n d'une relation et à éliminer les n-uplets en double apparaissant dans la nouvelle relation ; on la note $\Pi(A_1 \ldots A_n)R$.

Il s'agit d'une opération unaire essentielle dont la signature est :

relation × liste d'attributs → relation

En d'autres termes, la projection permet de choisir des colonnes dans le tableau. Si *R* est vide, la relation qui résulte de la projection est vide, mais pas forcément équivalente (elle contient généralement moins d'attributs).

Le tableau 3.7 montre un exemple de sélection.

Nom
Durand
Germain
Dupont

Tableau 3.7 : Exemple de projection sur la relation Personne du tableau 3.5 : Π(Nom)Personne

d. Union

Définition 28 -union- L'union est une opération portant sur deux relations R_1 et R_2 ayant le même schéma et construisant une troisième relation constituée des n-uplets appartenant à chacune des deux relations R_1 et R_2 sans doublon, on la note $R_1 \cup R_2$.

Il s'agit une opération binaire ensembliste commutative essentielle dont la signature est :

relation ∪ relation → relation

Comme nous l'avons déjà dit, R_1 et R_2 doivent avoir les mêmes attributs et si une même occurrence existe dans R_1 et R_2, elle n'apparaît qu'une seule fois dans le résultat de l'union. Le résultat de l'union est une nouvelle relation qui a les mêmes attributs que R_1 et R_2. Si R_1 et R_2 sont vides, la relation qui résulte de l'union est vide. Si R_1 (respectivement R_2) est vide, la relation qui résulte de l'union est identique à R_2 (respectivement R_1).

Le tableau 3.8 montre un exemple d'union.

Relation *R₁*		**Relation *R₂***		**Relation *R***	
Nom	**Prénom**	**Nom**	**Prénom**	**Nom**	**Prénom**
Durand	**Caroline**	**Dupont**	**Lisa**	**Durand**	**Caroline**
Germain	**Stan**	**Juny**	**Carole**	**Germain**	**Stan**
Dupont	**Lisa**	**Fourt**	**Lisa**	**Dupont**	**Lisa**
Germain	**Rose-Marie**			**Germain**	**Rose-Marie**
				Juny	**Carole**
				Fourt	**Lisa**

Tableau 3.8 : Exemple d'union : R = R1 ∪ R2

e. Intersection

Définition 29 -intersection- L'intersection est une opération portant sur deux relations R_1 et R_2 ayant le même schéma et construisant une troisième relation dont les n-uplets sont constitués de ceux appartenant aux deux relations, on la note $R_1 \cap R_2$.

Il s'agit une opération binaire ensembliste commutative dont la signature est :

relation ∩ relation → relation

Comme nous l'avons déjà dit, R1 et R2 doivent avoir les mêmes attributs. Le résultat de l'intersection est une nouvelle relation qui a les mêmes attributs que R1 et R2. Si R1 ou R2 ou les deux sont vides, la relation qui résulte de l'intersection est vide.

Le tableau 3.9 montre un exemple d'intersection.

Relation R_1			Relation R_2			Relation R	
Nom	**Prénom**		**Nom**	**Prénom**		**Nom**	**Prénom**
Durand	**Caroline**		**Dupont**	**Lisa**		**Durand**	**Caroline**
Germain	**Stan**		**Juny**	**Carole**		**Dupont**	**Lisa**
Dupont	**Lisa**		**Fourt**	**Lisa**		**Juny**	**Carole**
Germain	**Rose-Marie**		**Durand**	**Caroline**			
Juny	**Carole**						

Tableau 3.9 : Exemple d'intersection : R = R1 ∩ R2

f. Différence

Définition 30 -différence- La différence est une opération portant sur deux relations R_1 et R_2 ayant le même schéma et construisant une troisième relation dont les n-uplets sont constitués de ceux ne se trouvant que dans la relation R_1 ; on la note $R_1 - R_2$.

Il s'agit une opération binaire ensembliste non commutative essentielle dont la signature est :

relation − relation → relation

Comme nous l'avons déjà dit, R_1 et R_2 doivent avoir les mêmes attributs. Le résultat de la différence est une nouvelle relation qui a les mêmes attributs que R_1 et R_2. Si R_1 est vide, la relation qui résulte de la différence est vide. Si R_2 est vide, la relation qui résulte de la différence est identique à R_1.

Le tableau 3.10 montre un exemple de différence.

Relation R_1			Relation R_2			Relation R	
Nom	**Prénom**		**Nom**	**Prénom**		**Nom**	**Prénom**
Durand	**Caroline**		**Dupont**	**Lisa**		**Germain**	**Stan**
Germain	**Stan**		**Juny**	**Carole**		**Germain**	**Rose-Marie**
Dupont	**Lisa**		**Fourt**	**Lisa**			

Germain	**Rose-Marie**		**Durand**	**Caroline**			
Juny	**Carole**						

Tableau 3.10 : Exemple de différence : R = R1 − R2

g. Produit cartésien

Définition 31 -produit cartésien- Le produit cartésien est une opération portant sur deux relations R_1 et R_2 et qui construit une troisième relation regroupant exclusivement toutes les possibilités de combinaison des occurrences des relations R_1 et R_2, on la note $R_1 \times R_2$.

Il s'agit une opération binaire commutative essentielle dont la signature est :

relation × relation → relation

Le résultat du produit cartésien est une nouvelle relation qui a tous les attributs de R_1 et tous ceux de R_2. Si R_1 ou R_2 ou les deux sont vides, la relation qui résulte du produit cartésien est vide. Le nombre d'occurrences de la relation qui résulte du produit cartésien est le nombre d'occurrences de R_1 multiplié par le nombre d'occurrences de R_2.

Le tableau 3.11 montre un exemple de produit cartésien.

Relation *Amie*		**Relation *Cadeau***		**Relation *R***			
Nom	**Prénom**	**Article**	**Prix**	**Nom**	**Prénom**	**Article**	**Prix**
Fourt	**Lisa**	**livre**	**45**	**Fourt**	**Lisa**	**livre**	**45**
Juny	**Carole**	**poupée**	**25**	**Fourt**	**Lisa**	**poupée**	**25**
		montre	**87**	**Fourt**	**Lisa**	**montre**	**87**
				Juny	**Carole**	**livre**	**45**
				Juny	**Carole**	**poupée**	**25**
				Juny	**Carole**	**montre**	**87**

Tableau 3.11 : Exemple de produit cartésien : R = Amie × Cadeau

h. Jointure

Définition 32 -jointure- La jointure est une opération portant sur deux relations R_1 et R_2 qui construit une troisième relation regroupant exclusivement toutes les possibilités de combinaison des occurrences des relations R_1 et R_2 qui satisfont l'expression logique E. La jointure est notée $R_1 \triangleright\triangleleft E R_2$.

Il s'agit d'une opération binaire commutative dont la signature est :

relation × relation × expression logique → relation

Si R_1 ou R_2 ou les deux sont vides, la relation qui résulte de la jointure est vide.

En fait, la jointure n'est rien d'autre qu'un produit cartésien suivi d'une sélection :

$$R_1 \triangleright\triangleleft_E R_2 = \sigma_E (R_1 \times R_2)$$

Le tableau 3.12 montre un exemple de jointure.

Relation *Famille*				**Relation *Cadeau***				**Relation *R***					
Nom	**Prénom**	**Age**		**AgeC**	**Article**	**Prix**		**Nom**	**Prénom**	**Age**	**AgeC**	**Article**	**Prix**
Fourt	**Lisa**	**6**		**99**	**livre**	**30**		**Fourt**	**Lisa**	**6**	**99**	**livre**	**45**
Juny	**Carole**	**42**		**6**	**poupée**	**60**		**Fourt**	**Lisa**	**6**	**20**	**poupée**	**25**
Fidus	**Laure**	**16**		**20**	**baladeur**	**45**		**Fourt**	**Lisa**	**6**	**10**	**montre**	**87**
				10	**déguisement**	**15**		**Juny**	**Carole**	**42**	**99**	**livre**	**45**
								Fidus	**Laure**	**16**	**99**	**poupée**	**25**
								Fidus	**Laure**	**16**	**20**	**montre**	**87**

Tableau 3.12 : Exemple de jointure : R = Famille $\triangleright\triangleleft_{((Age \leq AgeC) \wedge (Prix < 50))}$ Cadeau

CHAPITRE III – INTRODUCTION AU SQL

1- PRESENTATION GENERALE

a. INTRODUCTION

Le langage SQL (Structured Query Language) peut être considéré comme le langage d'accès normalisé aux bases de données. Il est aujourd'hui supporté par la plupart des produits commerciaux que ce soit par les systèmes de gestion de bases de données micro tel que Access ou par les produits plus professionnels tels que Oracle. Il a fait l'objet de plusieurs normes ANSI/ISO dont la plus répandue aujourd'hui est la norme SQL2 qui a été définie en 1992.

Le succès du langage SQL est dû essentiellement à sa simplicité et au fait qu'il s'appuie sur le schéma conceptuel pour énoncer des requêtes en laissant le SGBD responsable de la stratégie d'exécution. Le langage SQL propose un langage de requêtes ensembliste et assertionnel. Néanmoins, le langage SQL ne possède pas la puissance d'un langage de programmation : entrées/sorties, instructions conditionnelles, boucles et affectations. Pour certains traitements il est donc nécessaire de coupler le langage SQL avec un langage de programmation plus complet.

De manière synthétique, on peut dire que SQL est un langage relationnel, il manipule donc des tables (i.e. des relations, c'est-à-dire des ensembles) par l'intermédiaire de requêtes qui produisent également des tables.

b. OBJECTIFS

- Créer la structure de la base de données et de ses tables
- Exécuter les taches de base de la gestion des données, telle que l'insertion, la modification et la suppression de données des tables
- Effectuer des requêtes simples ou complexes

2- CATEGORIE D'INSTRUCTIONS

a. LANGAGE DE DEFINITION DE DONNEES

Le **langage de définition de données** (LDD, ou Data Definition Language, soit DDL en anglais) est un langage orienté au niveau de la structure de la base de données. Le LDD permet de créer, modifier, supprimer des objets. Il permet également de définir le domaine des données (nombre, chaîne de caractères, date, booléen…) et d'ajouter des contraintes de valeur sur les données. Il permet enfin d'autoriser ou d'interdire l'accès aux données et d'activer ou de désactiver l'audit pour un utilisateur donné.

Les instructions du LDD sont : CREATE, ALTER, DROP, AUDIT, NOAUDIT, ANALYZE, RENAME, TRUNCATE.

b. LANGAGE DE MANIPULATION DES DONNEES

Le **langage de manipulation de données** (LMD, ou Data Manipulation Language, soit DML en anglais) est l'ensemble des commandes concernant la manipulation des données dans une base de données. Le LMD permet l'ajout, la suppression et la modification de lignes, la visualisation du contenu des tables et leur verrouillage.

Les instructions du LMD sont : INSERT, UPDATE, DELETE, SELECT, EXPLAIN, PLAN, LOCK TABLE.

CHAPITRE IV – LANGAGE DE DEFINITION DES DONNEES

1- CREATE

a. CREATE DATABASE

i. DESCRIPTION

La création d'une base de données en SQL est possible en ligne de commande. Même si les systèmes de gestion de base de données (SGBD) sont souvent utilisés pour créer une base, il convient de connaître la commande à utiliser, qui est très simple.

ii. SYNTAXE

Pour créer une base de données qui sera appelé « ma_base » il suffit d'utiliser la requête suivante qui est très simple :

```
CREATE DATABASE ma_base
```

b. CREATE TABLE

i. DESCRIPTION

La commande CREATE TABLE permet de créer une table en SQL. Un tableau est une entité qui est contenu dans une base de données pour stocker des données ordonnées dans des colonnes. La création d'une table sert à définir les colonnes et le type de données qui seront contenus dans chacun des colonne (entier, chaîne de caractères, date, valeur binaire …).

ii. SYNTAXE

La syntaxe générale pour créer une table est la suivante :

```
CREATE TABLE nom_de_la_table
(
    colonne1 type_donnees,
    colonne2 type_donnees,
    colonne3 type_donnees,
    colonne4 type_donnees
)
```

Dans cette requête, 4 colonnes ont été définies. Le mot-clé « type_donnees » sera à remplacer par un mot-clé pour définir le type de données (INT, DATE, TEXT …). Pour chaque colonne, il est également possible de définir des options telles que (liste non-exhaustive) :

- **NOT NULL** : empêche d'enregistrer une valeur nulle pour une colonne.
- **DEFAULT** : attribuer une valeur par défaut si aucune donnée n'est indiquée pour cette colonne lors de l'ajout d'une ligne dans la table.
- **PRIMARY KEY** : indiquer si cette colonne est considérée comme clé primaire pour un index.

iii. EXEMPLE

Imaginons que l'on souhaite créer une table Professeur, dans laquelle chaque ligne correspond à un professeur qui dispense les cours dans une Université. La requête pour créer cette table peut ressembler à ceci :

```
CREATE TABLE Professeur
(
    id INT PRIMARY KEY NOT NULL,
    nom VARCHAR(100),
    prenom VARCHAR(100),
    email VARCHAR(255),
    date_naissance DATE,
    numero VARCHAR(13),

)
```

c. TYPES DE DONNEES

Les types de données peuvent être :

INTEGER :

- Ce type permet de stocker des entiers signés codés sur 4 octets.

BIGINT :

- Ce type permet de stocker des entiers signés codés sur 8 octets.

REAL :

- Ce type permet de stocker des réels comportant 6 chiffres significatifs codés sur 4 octets.

DOUBLE PRÉCISION :

- Ce type permet de stocker des réels comportant 15 chiffres significatifs codés sur 8 octets.

NUMERIC[(précision, [longueur])] :

- Ce type de données permet de stocker des données numériques à la fois entières et réelles avec une précision de 1000 chiffres significatifs. longueur précise le nombre maximum de chiffres significatifs stockés et précision donne le nombre maximum de chiffres après la virgule.

CHAR(longueur) :

- Ce type de données permet de stocker des chaînes de caractères de longueur fixe. longueur doit être inférieur à 255, sa valeur par défaut est 1.

VARCHAR(longueur) :

- Ce type de données permet de stocker des chaînes de caractères de longueur variable. longueur doit être inférieur à 2000, il n'y a pas de valeur par défaut.

DATE :

- Ce type de données permet de stocker des données constituées d'une date.

TIMESTAMP :

- Ce type de données permet de stocker des données constituées d'une date et d'une heure.

BOOLEAN :

- Ce type de données permet de stocker des valeurs Booléenne.

MONEY :

- Ce type de données permet de stocker des valeurs monétaires.

TEXT :

- Ce type de données permet de stocker des chaînes de caractères de longueur variable.

d. CONTRAINTES D'INTEGRITE

NOT NULL ou NULL:

- Interdit (NOT NULL) ou autorise (NULL) l'insertion de valeur NULL pour cet attribut.

UNIQUE:

- Désigne l'attribut comme clé secondaire de la table. Deux n-uplets ne peuvent recevoir des valeurs identiques pour cet attribut, mais l'insertion de valeur NULL est toutefois autorisée. Cette contrainte peut apparaître plusieurs fois dans l'instruction.

PRIMARY KEY:

- Désigne l'attribut comme clé primaire de la table. La clé primaire étant unique, cette contrainte ne peut apparaître qu'une seule fois dans l'instruction. La définition d'une clé primaire composée se fait par l'intermédiaire d'une contrainte de table. En fait, la contrainte PRIMARY KEY est totalement équivalente à la contrainte UNIQUE NOT NULL.

CHECK (condition):

- Vérifie lors de l'insertion de n-uplets que l'attribut réalise la condition condition.

DEFAULT valeur:

- Permet de spécifier la valeur par défaut de l'attribut.

PRIMARY KEY (colonne…):

- Désigne la concaténation des attributs cités comme clé primaire de la table. Cette contrainte ne peut apparaître qu'une seule fois dans l'instruction.

UNIQUE (colonne…):

- Désigne la concaténation des attributs cités comme clé secondaire de la table. Dans ce cas, au moins une des colonnes participant à cette clé secondaire doit permettre de distinguer le n-uplet. Cette contrainte peut apparaître plusieurs fois dans l'instruction.

FOREIGN KEY (colonne…) REFERENCES table [(colonne…)] [ON DELETE CASCADE | SET NULL]:

- Contrainte d'intégrité référentielle pour un ensemble d'attributs de la table en cours de définition. Les valeurs prises par ces attributs doivent exister dans l'ensemble d'attributs spécifié et posséder une contrainte PRIMARY KEY ou UNIQUE dans la table table.

i. SYNTAXE

À la création d'une table, les contraintes d'intégrité se déclarent de la façon suivante :

```
CREATE TABLE nom_table (

nom_col_1 type_1 [CONSTRAINT nom_1_1] contrainte_1_1,
                 [CONSTRAINT nom_1_2] contrainte_1_2
 [CONSTRAINT nom_1] contrainte_1

)
```

ii. EXERCICE D'APPLICATION

1- Ci-après, on donne la représentation textuelle simplifiée d'une base de données intitulée "gestion_université".

— ETUDIANT(**CodeEt**, NomEt, PrenomEt, DatnEt)

—FILIERE(**CodeFil**, NomFil, DescFil)

—INSCRIPTION(**#CodeEt ,# CodeFil)**

—MATIERE(**CodeMat**, NomMat, CoefMat)

—COMPTER(**#CodeFil ,# CodeMat**)

— ENSEIGNANT(**CodeEns**, NomEns, PrenomEns, GradeEns)

—ENSEIGNER(**#CodeMat ,# CodeEns**)

—CLASSE(**#CodeCl** , DescCl)

—DISPENSER(**#CodeMat ,#CodeEns**,**#CodeCl,date**,Heure)

Créez toutes les tables du projets " gestion_université " en y spécifiant le types des données et les contraintes d'intégrité.

2- Soit la base de données intitulée "gestion_projet" permettant de gérer les projets relatifs au développement de logiciels.

Elle est décrite par la représentation textuelle simplifiée suivante :

— Developpeur (**NumDev**, NomDev, AdrDev, EmailDev, TelDev)

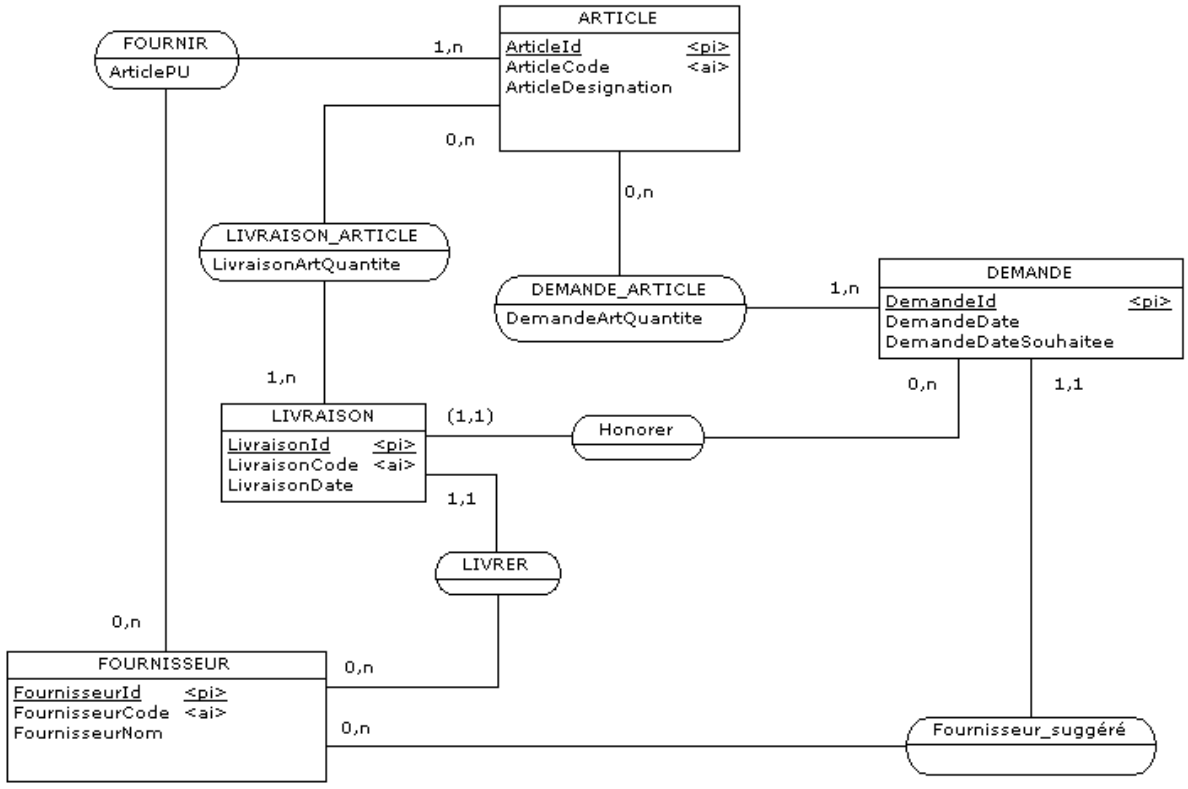
— Projet (**NumProj**, TitreProj, DateDeb, DateFin)

— Logiciel (**CodLog**, NomLog, PrixLog, #**NumProj**)

— Realisation (**#NumProj, #NumDev**)

Créez toutes les tables du projets "gestion_projet" en y spécifiant le types des données et les contraintes d'intégrité.

3- Ci-après, on donne la représentation graphique(MCD) d'une base de données concernant un stock de magasin.



Créez toutes les tables à partir du MCD en y spécifiant le types des données et les contraintes d'intégrité.

2- DROP

a. DROP DATABASE

i. DESCRIPTION

En SQL, la commande DROP DATABASE permet de supprimer totalement une base de données et tout ce qu'elle contient. Cette commande est à utiliser avec beaucoup d'attention car elle permet de supprimer tout ce qui est inclus dans une base: les tables, les données, les index …

ii. SYNTAXE

Pour supprimer la base de données « ma_base », la requête est la suivante :

```
DROP DATABASE ma_base
```

Attention : cela va supprimer toutes les tables et toutes les données de cette base. Si vous n'êtes pas sûr de ce que vous faites, n'hésitez pas à effectuer une sauvegarde de la base avant de supprimer.

b. DROP TABLE

i. DESCRIPTION

La commande DROP TABLE en SQL permet de supprimer définitivement une table d'une base de données. Cela supprime en même temps les éventuels index, trigger, contraintes et permissions associées à cette table.

Attention : il faut utiliser cette commande avec attention car une fois supprimée, les données sont perdues. Avant de l'utiliser sur une base importante il peut être judicieux d'effectuer un backup (une sauvegarde) pour éviter les mauvaises surprises.

ii. SYNTAXE

Pour supprimer une table « nom_table » il suffit simplement d'utiliser la syntaxe suivante :

```
DROP TABLE nom_table
```

A savoir : s'il y a une dépendance avec une autre table, il est recommandé de les supprimer avant de supprimer la table. C'est le cas par exemple s'il y a des clés étrangères.

Intérêts

Il arrive qu'une table soit créé temporairement pour stocker des données qui n'ont pas vocation à être réutiliser. La suppression d'une table non utilisée est avantageuse sur plusieurs aspects :

- Libérer de la mémoire et alléger le poids des backups
- Éviter des erreurs dans le futur si une table porte un nom similaire ou qui porte à confusion
- Lorsqu'un développeur ou administrateur de base de données découvre une application, il est plus rapide de comprendre le système s'il n'y a que les tables utilisées qui sont présente

iii. EXEMPLE

Imaginons qu'une base de données possède une table « client_2009 » qui ne sera plus jamais utilisé et qui existe déjà dans un ancien backup. Pour supprimer cette table, il suffit d'effectuer la requête suivante :

```
DROP TABLE client_2009
```

L'exécution de cette requête va permettre de supprimer la table.

3- ALTER

a. ALTER TABLE

i. DESCRIPTION

La commande ALTER TABLE en SQL permet de modifier une table existante. Il est ainsi possible d'ajouter une colonne, d'en supprimer une ou de modifier une colonne existante, par exemple pour changer le type.

ii. SYNTAXE

Syntaxe de base

D'une manière générale, la commande s'utilise de la manière suivante :

```
ALTER TABLE nom_table
instruction
```

Le mot-clé « instruction » ici sert à désigner une commande supplémentaire, qui sera détaillée ci-dessous selon l'action que l'on souhaite effectuer : ajouter, supprimer ou modifier une colonne.

Ajouter une colonne

Syntaxe

L'ajout d'une colonne dans une table est relativement simple et peut s'effectuer à l'aide d'une requête ressemblant à ceci :

```
ALTER TABLE nom_table
ADD nom_colonne type_donnees
```

Exemple

Pour ajouter une colonne qui correspond à une rue sur une table utilisateur, il est possible d'utiliser la requête suivante :

```
ALTER TABLE nom_table
DROP adresse_rue VARCHAR(255)
```

Supprimer une colonne

Une syntaxe permet également de supprimer une colonne pour une table. Il y a 2 manières totalement équivalente pour supprimer une colonne :

```
ALTER TABLE nom_table
DROP nom_colonne
```

Ou (le résultat sera le même)

```
ALTER TABLE nom_table
DROP COLUMN nom_colonne
```

Modifier une colonne

Pour modifier une colonne, comme par exemple changer le type d'une colonne, il y a différentes syntaxes selon le SGBD.

MySQL

Pour MySQL, il faut également indiquer le type de la colonne.

```
ALTER TABLE nom_table
MODIFY nom_colonne type_donnees
```

PostgreSQL

```
ALTER TABLE nom_table
ALTER COLUMN nom_colonne TYPE type_donnees
```

Ici, le mot-clé « type_donnees » est à remplacer par un type de données tel que INT, VARCHAR, TEXT, DATE …

Renommer une colonne

Pour renommer une colonne, il convient d'indiquer l'ancien nom de la colonne et le nouveau nom de celle-ci.

MySQL

Pour MySQL, il faut également indiquer le type de la colonne.

```
ALTER TABLE nom_table
CHANGE colonne_ancien_nom colonne_nouveau_nom type_donnees
```

Ici « type_donnees » peut correspondre par exemple à INT, VARCHAR, TEXT, DATE …

PostgreSQL

Pour PostgreSQL la syntaxe est plus simple et ressemble à ceci (le type n'est pas demandé) :

```
ALTER TABLE nom_table
RENAME COLUMN colonne_ancien_nom TO colonne_nouveau_nom
```

Ajouter une contrainte

La syntaxe de déclaration de contrainte est identique à celle vue lors de la création de tables.

```
ALTER TABLE nom_table ADD [CONSTRAINT nom_contrainte] contrainte
```

Supprimer une contrainte

```
ALTER TABLE nom_table DROP CONSTRAINT nom_contrainte
```

b. EXERCICE D'APPLICATION

1- Ci-après, on donne la représentation textuelle simplifiée De la base de données intitulée "gestion_université".

— ETUDIANT(**CodeEt**, NomEt, PrenomEt, DatnEt)

—FILIERE(**CodeFil**, NomFil, DescFil)

—INSCRIPTION(**#CodeEt ,# CodeFil)**

—MATIERE(**CodeMat**, NomMat, CoefMat)

—COMPTER(**#CodeFil ,# CodeMat**)

— ENSEIGNANT(**CodeEns**, NomEns, PrenomEns, GradeEns)

—ENSEIGNER(**#CodeMat ,# CodeEns**)

—CLASSE(**#CodeCl** , DescCl)

—DISPENSER(**#CodeMat ,#CodeEns**,**#CodeCl,date**,Heure)

Apres révision de la base de donnes, il a été convenu qu'en fin de cycle de formation, une note par matière, est attribuée à chaque étudiant et chaque étudiant doit fournir un numéro de contact unique.

— NOTE(#CodeEt, #CodeMat, note)

— ETUDIANT(**CodeEt**, NomEt, PrenomEt, DatnEt, NumeroEt)

1- Donner la requête SQL pour créer la table NOTE puis celle pour modifier la table et y spécifier la clé primaire et les clés étrangères.
2- Donner la requête SQL pour Modifier la table ETUDIANT pour y ajouter la colonne NumeroEt
3- Donner la requête SQL pour Modifier la table ETUDIANT pour y ajouter la contrainte UNIQUE sur le colonne NumeroEt

2- On considère la base de données BD_AIR_MAROC suivante :

— PILOTE (NUMPIL, NOMPIL, VILLE, SALAIRE)
— AVION (NUMAV, NOMAV, CAPACITE, VILLE)
— VOL (NUMVOL, VILLE_DEP, VILLE_ARR, H_DEP, H_ARR)

1- Donner la requête SQL pour ajouter leur clé primaire a toutes les tables
2- Donner la requête SQL pour ajouter la clé étrangère de la table PILOTE a la table VOL
3- Donner la requête SQL pour ajouter la clé étrangère de la table AVION a la table VOL
4- Supprimer les deux clés étrangères de la table VOL.

CHAPITRE V – LANGAGE DE MANIPULATION DES DONNEES

1- INSERT

a. DESCRIPTION

L'insertion de données dans une table s'effectue à l'aide de la commande INSERT INTO. Cette commande permet au choix d'inclure une seule ligne à la base existante ou plusieurs lignes d'un coup.

b. SYNTAXE

INSERTION D'UNE LIGNE A LA FOIS

Pour insérer des données dans une base, il y a 2 syntaxes principales :

- Insérer une ligne en indiquant les informations pour chaque colonne existante (en respectant l'ordre)
- Insérer une ligne en spécifiant les colonnes que vous souhaitez compléter. Il est possible d'insérer une ligne en renseigner seulement une partie des colonnes

INSERER UNE LIGNE EN SPECIFIANT TOUTES LES COLONNES

La syntaxe pour remplir une ligne avec cette méthode est la suivante :

```
INSERT INTO table
VALUES ('valeur 1', 'valeur 2', ...)
```

Cette syntaxe possède les avantages et inconvénients suivants :

- Obliger de remplir toutes les données, tout en respectant l'ordre des colonnes
- Il n'y a pas le nom de colonne, donc les fautes de frappe sont limitées. Par ailleurs, les colonnes peuvent être renommées sans avoir à changer la requête
- L'ordre des colonnes doit rester identique sinon certaines valeurs prennent le risque d'être complétée dans la mauvaise colonne

INSERER UNE LIGNE EN SPECIFIANT SEULEMENT LES COLONNES SOUHAITEES

Cette deuxième solution est très similaire, excepté qu'il faut indiquer le nom des colonnes avant « VALUES ». La syntaxe est la suivante :

```
INSERT INTO table
(nom_colonne_1, nom_colonne_2, ...
VALUES ('valeur 1', 'valeur 2', ...)
```

A noter : il est possible de ne pas renseigner toutes les colonnes. De plus, l'ordre des colonnes n'est pas important.

Insertion de plusieurs lignes à la fois

Il est possible d'ajouter plusieurs lignes à un tableau avec une seule requête. Pour ce faire, il convient d'utiliser la syntaxe suivante :

```
INSERT INTO client (prenom, nom, ville, age)
VALUES
('Rébecca', 'Armand', 'Saint-Didier-des-Bois', 24),
('Aimée', 'Hebert', 'Marigny-le-Châtel', 36),
('Marielle', 'Ribeiro', 'Maillères', 27) ;
```

A noter : lorsque le champ à remplir est de type VARCHAR ou TEXT il faut indiquer le texte entre guillemet simple. En revanche, lorsque la colonne est un numérique tel que INT ou BIGINT il n'y a pas besoin d'utiliser de guillemet, il suffit juste d'indiquer le nombre.
Un tel exemple sur une table vide va créer le tableau suivant :

id	prenom	nom	ville	age
1	Rébecca	Armand	Saint-Didier-des-Bois	24
2	Aimée	Hebert	Marigny-le-Châtel	36
3	Marielle	Ribeiro	Maillères	27

c. EXEMPLE

Ci-après, on donne la représentation textuelle simplifiée De la base de données intitulée "gestion_université".

— ETUDIANT(**CodeEt**, NomEt, PrenomEt, DatnEt)

—FILIERE(**CodeFil**, NomFil, DescFil)

—INSCRIPTION(**#CodeEt ,# CodeFil)**

Inscrivez deux étudiant dans la filière GCL.

2- UPDATE

a. DESCRIPTION

La commande UPDATE permet d'effectuer des modifications sur des lignes existantes. Très souvent cette commande est utilisée avec WHERE pour spécifier sur quelles lignes doivent porter la ou les modifications.

b. SYNTAXE

La syntaxe basique d'une requête utilisant UPDATE est la suivante :

```
UPDATE table
SET nom_colonne_1 = 'nouvelle valeur'
WHERE condition
```

Cette syntaxe permet d'attribuer une nouvelle valeur à la colonne nom_colonne_1 pour les lignes qui respectent la condition stipulée avec WHERE. Il est aussi possible d'attribuer la même valeur à la colonne nom_colonne_1 pour toutes les lignes d'une table si la condition WHERE n'était pas utilisée.

A noter, pour spécifier en une seule fois plusieurs modification, il faut séparer les attributions de valeur par des virgules. Ainsi la syntaxe deviendrait la suivante :

```
UPDATE table
SET colonne_1 = 'valeur 1', colonne_2 = 'valeur 2', colonne_3 = 'valeur 3'
WHERE condition
```

c. EXEMPLE

id	nom	rue	ville	code_postal	pays
1	Chantal	2 Avenue du Petit Trianon	Puteaux	92800	France
2	Pierre	8 Rue de l'Allier	Ponthion	51300	France
3	Romain	3 Chemin du Chiron	Trévérien	35190	France

Imaginons une table « client » qui présente les coordonnées de clients.

Pour modifier l'adresse du client Pierre, il est possible d'utiliser la requête suivante :

```
UPDATE client
  SET rue = '49 Rue Ameline',
  ville = 'Saint-Eustache-la-Forêt',
  code_postal = '76210'
WHERE id = 2
```

Résultat :

id	nom	rue	ville	code_postal	pays
1	Chantal	2 Avenue du Petit Trianon	Puteaux	92800	France
2	Pierre	49 Rue Ameline	Saint-Eustache-la-Forêt	76210	France
3	Romain	3 Chemin du Chiron	Trévérien	35190	France

3- DELETE

a. DESCRIPTION

La commande DELETE en SQL permet de supprimer des lignes dans une table. En utilisant cette commande associée à WHERE il est possible de sélectionner les lignes concernées qui seront supprimées.

Attention : Avant d'essayer de supprimer des lignes, il est recommandé d'effectuer une sauvegarde de la base de données, ou tout du moins de la table concernée par la suppression. Ainsi, s'il y a une mauvaise manipulation il est toujours possible de restaurer les données.

b. SYNTAXE

La syntaxe pour supprimer des lignes est la suivante :

```
DELETE FROM table
WHERE condition
```

Attention : s'il n'y a pas de condition WHERE alors toutes les lignes seront supprimées et la table sera alors vide.

c. EXEMPLE

Imaginons une table « utilisateur » qui contient des informations sur les utilisateurs d'une application.

id	nom	prenom	date_inscription
1	Bazin	Daniel	2012-02-13
2	Favre	Constantin	2012-04-03

3	Clerc	Guillaume	2012-04-12
4	Ricard	Rosemonde	2012-06-24
5	Martin	Natalie	2012-07-02

Si l'on souhaite supprimer les utilisateurs qui se sont inscrit avant le « 10/04/2012", il va falloir effectuer la requête suivante :

```
DELETE FROM utilisateur
WHERE date_inscription < '2012-04-10'
```

La requête permettra alors de supprimer les utilisateurs « Daniel » et « Constantin ». La table contiendra alors les données suivantes:

id	nom	prenom	date_inscription
3	Clerc	Guillaume	2012-04-12
4	Ricard	Rosemonde	2012-06-24
5	Martin	Natalie	2012-07-02

Il ne faut pas oublier qu'il est possible d'utiliser d'autres conditions pour sélectionner les lignes à supprimer.

4- TRUNCATE

a. DESCRIPTION

En SQL, la commande TRUNCATE permet de supprimer toutes les données d'une table sans supprimer la table en elle-même. En d'autres mots, cela permet de purger la table. Cette instruction diffère de la commande DROP qui a pour but de supprimer les données ainsi que la table qui les contient.

A noter : l'instruction TRUNCATE est semblable à l'instruction DELETE sans utilisation de WHERE. Parmi les petite différences TRUNCATE est toutefois plus rapide et utilise moins de ressource. Ces gains en performance se justifie notamment parce que la requête n'indiquera pas le nombre d'enregistrement supprimés et qu'il n'y aura pas d'enregistrement des modifications dans le journal.

b. SYNTAXE

Cette instruction s'utilise dans une requête SQL semblable à celle-ci :

```
TRUNCATE TABLE `table`
```

Dans cet exemple, les données de la table « table » seront perdues une fois cette requête exécutée.

c. EXEMPLE

Pour montrer un exemple concret de l'utilisation de cette commande, nous pouvons imaginez un système informatique contenant la liste des fournitures d'une entreprise. Ces données seraient tout simplement stockées dans une table « fourniture ».

Table « fourniture » :

id	nom	date_ajout
1	Ordinateur	2013-04-05
2	Chaise	2013-04-14
3	Bureau	2013-07-18
4	Lampe	2013-09-27

Il est possible de supprimer toutes les données de cette table en utilisant la requête suivante :

```
TRUNCATE TABLE `fourniture`
```

Une fois la requête exécutée, la table ne contiendra plus aucun enregistrement. En d'autres mots, toutes les lignes du tableau présenté ci-dessus auront été supprimées.

5- SELECT

a. SELECT

i. DESCRIPTION

L'utilisation la plus courante de SQL consiste à lire des données issues de la base de données. Cela s'effectue grâce à la commande SELECT, qui retourne des enregistrements dans un tableau de résultat. Cette commande peut sélectionner une ou plusieurs colonnes d'une table.

ii. SYNTAXE

L'utilisation basique de cette commande s'effectue de la manière suivante :

```
SELECT nom_du_champ
FROM nom_du_tableau
```

Cette requête va sélectionner (SELECT) le champ « nom_du_champ » provenant (FROM) du tableau appelé « nom_du_tableau ».

iii. EXEMPLE

Imaginons une base de données appelée « client » qui contient des informations sur les clients d'une entreprise.

Table « client » :

identifiant	prenom	nom	ville
1	Pierre	Dupond	Paris
2	Sabrina	Durand	Nantes
3	Julien	Martin	Lyon
4	David	Bernard	Marseille
5	Marie	Leroy	Grenoble

Si l'on veut avoir la liste de toutes les villes des clients, il suffit d'effectuer la requête suivante :

```
SELECT ville
FROM client
```

Résultat :

ville
Paris
Nantes
Lyon
Marseille
Grenoble

Obtenir plusieurs colonnes

Avec la même table client il est possible de lire plusieurs colonnes à la fois. Il suffit tout simplement de séparer les noms des champs souhaités par une virgule. Pour obtenir les prénoms et les noms des clients il faut alors faire la requête suivante:

```
SELECT prenom, nom
FROM client
```

Résultat :

prenom	nom
Pierre	Dupond
Sabrina	Durand
Julien	Martin
David	Bernard
Marie	Leroy

Obtenir toutes les colonnes d'un tableau

Il est possible de retourner automatiquement toutes les colonnes d'un tableau sans avoir à connaître le nom de toutes les colonnes. Au lieu de lister toutes les colonnes, il faut simplement utiliser le caractère « * » (étoile). C'est un joker qui permet de sélectionner toutes les colonnes. Il s'utilise de la manière suivante :

```
SELECT * FROM client
```

Cette requête retourne exactement les mêmes colonnes qu'il y a dans la base de données. Dans notre cas, le résultat sera donc :

identifiant	prenom	nom	ville
1	Pierre	Dupond	Paris
2	Sabrina	Durand	Nantes
3	Julien	Martin	Lyon
4	David	Bernard	Marseille
5	Marie	Leroy	Grenoble

Il y a des avantages et des inconvénients à l'utiliser. Pour en savoir plus sur le sujet il est recommandé de lire l'article avantage et inconvénient du sélecteur étoile.

Un requête SELECT peut devenir assez longue. Juste à titre informatif, voici une requête SELECT qui possède presque toutes les commandes possibles :

```
SELECT *
FROM table
WHERE condition
GROUP BY expression
HAVING condition
{ UNION | INTERSECT | EXCEPT }
ORDER BY expression
LIMIT count
```

A noter : cette requête imaginaire sert principale d'aide-mémoire pour savoir dans quel ordre sont utilisé chacun des commandes au sein d'une requête SELECT.

b. DISTINCT

i. DESCRIPTION

L'utilisation de la commande SELECT en SQL permet de lire toutes les données d'une ou plusieurs colonnes. Cette commande peut potentiellement afficher des lignes en doubles. Pour éviter des redondances dans les résultats il faut simplement ajouter DISTINCT après le mot SELECT.

ii. SYNTAXE

L'utilisation basique de cette commande consiste alors à effectuer la requête suivante :

```
SELECT DISTINCT ma_colonne
FROM nom_du_tableau
```

Cette requête sélectionne le champ « ma_colonne » de la table « nom_du_tableau » en évitant de retourner des doublons.

iii. EXEMPLE

Prenons le cas concret d'une table « client » qui contient des noms et prénoms :

identifiant	prenom	nom
1	Pierre	Dupond
2	Sabrina	Bernard
3	David	Durand
4	Pierre	Leroy
5	Marie	Leroy

En utilisant seulement SELECT tous les noms sont retournés, or la table contient plusieurs fois le même prénom (cf. Pierre). Pour sélectionner uniquement les prénoms uniques il faut utiliser la requête suivante :

```
SELECT DISTINCT prenom
FROM client
```

Résultat :

prenom
Pierre
Sabrina
David

Marie

Ce résultat affiche volontairement qu'une seule fois le prénom « Pierre » grâce à l'utilisation de la commande DISTINCT qui n'affiche que les résultats distincts.

Intérêt

L'utilisation de la commande DISTINCT est très pratique pour éviter les résultats en doubles. Cependant, pour optimiser les performances il est préférable d'utiliser la commande SQL GROUP BY lorsque c'est possible.

c. AS (alias)

i. DESCRIPTION

Dans le langage SQL il est possible d'utiliser des alias pour renommer temporairement une colonne ou une table dans une requête. Cette astuce est particulièrement utile pour faciliter la lecture des requêtes.

Intérêts et utilités :

Alias sur une colonne

Permet de renommer le nom d'une colonne dans les résultats d'une requête SQL. C'est pratique pour avoir un nom facilement identifiable dans une application qui doit ensuite exploiter les résultats d'une recherche.

Cas concrets d'utilisations :

- Une colonne qui s'appelle normalement c_iso_3166 peut être renommée « code_pays » (cf. le code ISO 3166 correspond au code des pays), ce qui est plus simple à comprendre dans le reste du code par un développeur.
- Une requête qui utilise la commande UNION sur des champs aux noms différents peut être ambigu pour un développeur. En renommant les champs avec un même nom il est plus simple de traiter les résultats.
- Lorsqu'une fonction est utilisée, le nom d'une colonne peut-être un peu complexe. Il est ainsi possible de renommer la colonne sur laquelle il y a une fonction SQL. Exemple : SELECT COUNT(*) AS nombre_de_resultats FROM `table`.
- Lorsque plusieurs colonnes sont combinées il est plus simple de renommer la nouvelle colonne qui est une concaténation de plusieurs champs.

Alias sur une table

Permet d'attribuer un autre nom à une table dans une requête SQL. Cela peut aider à avoir des noms plus courts, plus simple et plus facilement compréhensible. Ceci est particulièrement vrai lorsqu'il y a des jointures.

ii. SYNTAXE

Alias sur une colonne

La syntaxe pour renommer une colonne de colonne1 à c1 est la suivante :

```
SELECT colonne1 AS c1, colonne2
FROM `table`
```

Cette syntaxe peut également s'afficher de la façon suivante :

```
SELECT colonne1 c1, colonne2
FROM `table`
```

A noter : à choisir il est préférable d'utiliser la commande « AS » pour que ce soit plus explicite (plus simple à lire qu'un simple espace), d'autant plus que c'est recommandé dans le standard ISO pour concevoir une requête SQL.

Alias sur une table

La syntaxe pour renommer une table dans une requête est la suivante :

```
SELECT *
FROM `nom_table` AS t1
```

d. WHERE

i. DESCRIPTION

La commande WHERE dans une requête SQL permet d'extraire les lignes d'une base de données qui respectent une condition. Cela permet d'obtenir uniquement les informations désirées.

ii. SYNTAXE

La commande WHERE s'utilise en complément à une requête utilisant SELECT. La façon la plus simple de l'utiliser est la suivante :

```
SELECT nom_colonnes
FROM nom_table
WHERE condition
```

iii. EXEMPLE

Imaginons une base de données appelée « client » qui contient le nom des clients, le nombre de commandes qu'ils ont effectués et leur ville :

id	nom	nbr_commande	ville
1	Paul	3	paris
2	Maurice	0	rennes
3	Joséphine	1	toulouse
4	Gérard	7	paris

Pour obtenir seulement la liste des clients qui habitent à Paris, il faut effectuer la requête suivante :

```
SELECT *
FROM client
WHERE ville = 'paris'
```

Résultat :

id	nom	nbr_commande	nbr_commande
1	Paul	3	paris
4	Gérard	7	paris

OPERATEURS DE COMPARAISONS

Il existe plusieurs opérateurs de comparaisons. La liste ci-jointe présente quelques-uns des opérateurs les plus couramment utilisés.

Opérateur	Description
=	Égale
<>	Pas égale
!=	Pas égale
>	Supérieur à
<	Inférieur à
>=	Supérieur ou égale à
<=	Inférieur ou égale à
IN	Liste de plusieurs valeurs possibles
BETWEEN	Valeur comprise dans un intervalle donnée (utile pour les nombres ou dates)
LIKE	Recherche en spécifiant le début, milieu ou fin d'un mot.
IS NULL	Valeur est nulle

IS NOT NULL	Valeur n'est pas nulle

e. AND & OR

i. DESCRIPTION

Une requête SQL peut être restreinte à l'aide de la condition WHERE. Les opérateurs logiques AND et OR peuvent être utilisées au sein de la commande WHERE pour combiner des conditions.

ii. SYNTAXE

Les opérateurs sont à ajoutés dans la condition WHERE. Ils peuvent être combinés à l'infini pour filtrer les données comme souhaités.

L'opérateur AND permet de s'assurer que la condition1 ET la condition2 sont vrai :

```
SELECT nom_colonnes
FROM nom_table
WHERE condition1 AND condition2
```

L'opérateur OR vérifie quant à lui que la condition1 OU la condition2 est vrai :

```
SELECT nom_colonnes FROM nom_table
WHERE condition1 OR condition2
```

Ces opérateurs peuvent être combinés à l'infini et mélangés. L'exemple ci-dessous filtre les résultats de la table « nom_table » si condition1 ET condition2 OU condition3 est vrai :

```
SELECT nom_colonnes FROM nom_table
WHERE condition1 AND (condition2 OR condition3)
```

Attention : il faut penser à utiliser des parenthèses lorsque c'est nécessaire. Cela permet d'éviter les erreurs car et ça améliore la lecture d'une requête par un humain.

iii. EXEMPLE

Pour illustrer les prochaines commandes, nous allons considérer la table « produit » suivante :

id	nom	categorie	stock	prix
1	ordinateur	informatique	5	950
2	clavier	informatique	32	35
3	souris	informatique	16	30
4	crayon	fourniture	147	2

Opérateur AND

L'opérateur AND permet de joindre plusieurs conditions dans une requête. En gardant la même table que précédemment, pour filtrer uniquement les produits informatiques qui sont presque en rupture de stock (moins de 20 produits disponible) il faut exécuter la requête suivante :

```
SELECT * FROM produit
WHERE categorie = 'informatique' AND stock < 20
```

Résultat :

id	nom	categorie	stock	prix
1	ordinateur	informatique	5	950
3	souris	informatique	16	30

Opérateur OR

Pour filtrer les données pour avoir uniquement les données sur les produits « ordinateur » ou « clavier » il faut effectuer la recherche suivante **:**

```
SELECT * FROM produit
WHERE nom = 'ordinateur' OR nom = 'clavier'
```

Résultats :

id	nom	categorie	stock	prix
1	ordinateur	informatique	5	950
2	clavier	informatique	32	35

Combiner AND et OR

Il ne faut pas oublier que les opérateurs peuvent être combinés pour effectuer de puissante recherche. Il est possible de filtrer les produits « informatique » avec un stock inférieur à 20 et les produits « fourniture » avec un stock inférieur à 200 avec la recherche suivante :

```
SELECT * FROM produit
WHERE ( categorie = 'informatique' AND stock < 20 )
OR ( categorie = 'fourniture' AND stock < 200 )
```

Résultats :

id	nom	categorie	stock	prix
1	ordinateur	informatique	5	950
2	clavier	informatique	32	35
4	crayon	fourniture	147	2

EXERCICE D'APPLICATION (*)

1- On considère la base de données BD_AIR_MAROC suivante :

— PILOTE (**NUMPIL**, NOMPIL, VILLE, SALAIRE)

— AVION (**NUMAV**, NOMAV, CAPACITE, VILLE)

— VOL (**NUMVOL**, #**NUMPIL**, #**NUMAV**, VILLE_DEP, VILLE_ARR, H_DEP, H_ARR)

1. Donnez la liste des avions dont la capacité est supérieure à 350 passagers.

2. Quels sont les numéros et noms des avions localisés à Marrakech ?

3. Quels sont les numéros des pilotes en service et les villes de départ de leurs vols ?

4. Donnez toutes les informations sur les pilotes de la compagnie.

5. Quel est le nom des pilotes domiciliés à Meknès dont le salaire est supérieur à 20000 DH?

6. Quels sont les avions (numéro et nom) localisés à Marrakech ou dont la capacité est inférieure à 350 passagers ?

7. Donnez le numéro des vols effectués au départ de Marrakech par des pilotes de Meknès ?

8. Quels sont les vols effectués par un avion qui n'est pas localisé à Marrakech ?

10. Quelles sont les villes desservies à partir de la ville d'arrivée d'un vol au départ de Guelmim ?

2- Soit le schéma relationnel suivant :

— Departement (**N_Dep**, NomD, Directeur)

— Employe (**Matricule**, Nom, Prénom, DateNaissance, Adresse, Salaire, #N_dep, superieur)

— Projet (**N_pro**, NomP, Lieu, **#N_Dep**)

— Travaille (**#Matricule, #N_Proj**, Heures)

L'attribut supérieur dans la relation Employe contient le matricule du supérieur direct de l'employé. Chaque employé appartient à un département et travaille sur zéro, un ou plusieurs projets. Chaque projet est rattaché à un département qui peut être différent de celui des employés travaillant sur ce projet. Exprimer en SQL les requêtes suivantes :

1. Date de naissance et l'adresse de Taha Lamharchi.

2. Nom et adresse des employés qui travaillent au département de recherche.

Ing. KPELOU Manzamasso

3. Nom et Prénom des employés dont le supérieur est Taha Lamharchi.

4. Nom des employés qui travaillent plus de 10heures sur un projet à Guelmim

5. Nom et prénom des employés qui ne travaillent sur aucun projet.

6. Nom des employés qui ne travaillent pas sur un projet à Guelmim.

f. IN

i. DESCRIPTION

L'opérateur logique IN dans SQL s'utilise avec la commande WHERE pour vérifier si une colonne est égale à une des valeurs comprises dans set de valeurs déterminés. C'est une méthode simple pour vérifier si une colonne est égale à une valeur OU une autre valeur OU une autre valeur et ainsi de suite, sans avoir à utiliser de multiple fois l'opérateur OR.

ii. SYNTAXE

Pour chercher toutes les lignes où la colonne « nom_colonne » est égale à 'valeur 1' OU 'valeur 2' ou 'valeur 3', il est possible d'utiliser la syntaxe suivante :

```
SELECT nom_colonne
FROM table
WHERE nom_colonne IN ( valeur1, valeur2, valeur3, ... )
```

A savoir : entre les parenthèses il n'y a pas de limite du nombre d'arguments. Il est possible d'ajouter encore d'autres valeurs.

Cette syntaxe peut être associée à l'opérateur NOT pour recherche toutes les lignes qui ne sont pas égales à l'une des valeurs stipulées.

Simplicité de l'opérateur IN

La syntaxe utilisée avec l'opérateur est plus simple que d'utiliser une succession d'opérateur OR. Pour le montrer concrètement avec un exemple, voici 2 requêtes qui retournerons les mêmes résultats, l'une utilise l'opérateur IN, tandis que l'autre utilise plusieurs OR.

Requête avec plusieurs OR

```
SELECT prenom
FROM utilisateur
WHERE prenom = 'Maurice' OR prenom = 'Marie' OR prenom = 'Thimoté'
```

Requête équivalent avec l'opérateur IN

```
SELECT prenom
FROM utilisateur
WHERE prenom IN ( 'Maurice', 'Marie', 'Thimoté' )
```

iii. EXEMPLE

Imaginons une table « adresse » qui contient une liste d'adresse associée à des utilisateurs d'une application.

id	id_utilisateur	addr_rue	addr_code_postal	addr_ville
1	23	35 Rue Madeleine Pelletier	25250	Bournois
2	43	21 Rue du Moulin Collet	75006	Paris
3	65	28 Avenue de Cornouaille	27220	Mousseaux-Neuville
4	67	41 Rue Marcel de la Provoté	76430	Graimbouville
5	68	18 Avenue de Navarre	75009	Paris

Si l'ont souhaite obtenir les enregistrements des adresses de Paris et de Graimbouville, il est possible d'utiliser la requête suivante :

```
SELECT *
FROM adresse
WHERE addr_ville IN ( 'Paris', 'Graimbouville' )
```

Résultats :

id	id_utilisateur	addr_rue	addr_code_postal	addr_ville
2	43	21 Rue du Moulin Collet	75006	Paris
4	67	41 Rue Marcel de la Provoté	76430	Graimbouville
5	68	18 Avenue de Navarre	75009	Paris

EXERCICE D'APPLICATION (*1)

1- Soit le schéma relationnel suivant qui représente la base de données d'une agence de voyage en ligne.

— CLIENT (**NumCli**, Nom, Prénom, e-mail, NumCB)

— VOYAGE (**CodeVoyage**, Destination, Durée, Prix)

— RESERVATION **(#NumCli, #CodeVoyage**, DateRes)

Formuler en SQL les requêtes suivantes :

1. Nom, prénom et e-mail des clients ayant une réservation en cours

2. Nom, prénom et e-mail des clients n'ayant aucune réservation en cours

3. Destination et liste des clients ayant réservés pour un voyage de plus de 10 jours et coûtant moins de 1000 DH.

2- Soit la base de données « cinéma » dont le schéma relationnel est donné ci-dessous :

— VILLE (CodePostal, NomVille)

— CINEMA (**NumCine**, NomCine, Adresse, #**CodePostal**)

— SALLE (**NumSalle**, Capacité, #**NumCine**)

— FILM (**NumExploit**, Titre, Durée)

— PROJECTION (**#NumExploit, #NumSalle**, NumSemaine, Nbentrees)

Ecrivez les requêtes suivantes en algèbre relationnelle :

1. Titre des films dont la durée est supérieure ou égale à deux heures

2. Nom des villes abritant un cinéma nommé « RIF »

3. Nom des cinémas situés à Meknès ou contenant au moins une salle de plus 100 places

4. Nom, adresse et ville des cinémas dans lesquels on joue le film « Hypnose » la semaine 18

5. Titre des films qui n'ont pas été projetés

g. BETWEEN

i. DESCRIPTION

L'opérateur BETWEEN est utilisé dans une requête SQL pour sélectionner un intervalle de données dans une requête utilisant WHERE. L'intervalle peut être constitué de chaînes de caractères, de nombres ou de dates. L'exemple le plus concret consiste par exemple à récupérer uniquement les enregistrements entre 2 dates définies.

ii. SYNTAXE

L'utilisation de la commande BETWEEN s'effectue de la manière suivante :

```
SELECT *
FROM table
WHERE nom_colonne BETWEEN 'valeur1' AND 'valeur2'
```

La requête suivante retournera toutes les lignes dont la valeur de la colonne « nom_colonne » sera comprise entre valeur1 et valeur2.

iii. EXEMPLE

Imaginons une table « utilisateur » qui contient les membres d'une application en ligne.

id	nom	date_inscription
1	Maurice	2012-03-02
2	Simon	2012-03-05
3	Chloé	2012-04-14
4	Marie	2012-04-15
5	Clémentine	2012-04-26

Si l'on souhaite obtenir les membres qui se sont inscrit entre le 1 avril 2012 et le 20 avril 2012 il est possible d'effectuer la requête suivante :

```
SELECT *
FROM utilisateur
WHERE date_inscription BETWEEN '2012-04-01' AND '2012-04-20'
```

Résultat :

id	nom	date_inscription
3	Chloé	2012-04-14
4	Marie	2012-04-15

Si l'ont souhaite obtenir tous les résultats dont l'identifiant n'est pas situé entre 4 et 10, il faudra alors utiliser la requête suivante :

```
SELECT *
FROM utilisateur
WHERE id NOT BETWEEN 4 AND 10
```

Résultat :

id	nom	date_inscription
1	Maurice	2012-03-02
2	Simon	2012-03-05
3	Chloé	2012-04-14

h. LIKE

i. DESCRIPTION

L'opérateur LIKE est utilisé dans la clause WHERE des requêtes SQL. Ce mot-clé permet d'effectuer une recherche sur un modèle particulier. Il est par exemple possible de rechercher les enregistrements dont la valeur d'une colonne commence par telle ou telle lettre. Les modèles de recherches sont multiples.

ii. SYNTAXE

La syntaxe à utiliser pour utiliser l'opérateur LIKE est la suivante :

```
SELECT *
FROM table
WHERE colonne LIKE modele
```

Dans cet exemple le « modèle » n'a pas été défini, mais il ressemble très généralement à l'un des exemples suivants :

- **LIKE '%a' :** le caractère « % » est un caractère joker qui remplace tous les autres caractères. Ainsi, ce modèle permet de rechercher toutes les chaines de caractère qui se termine par un « a ».
- **LIKE 'a%' :** ce modèle permet de rechercher toutes les lignes de « colonne » qui commence par un « a ».
- **LIKE '%a%' :** ce modèle est utilisé pour rechercher tous les enregistrements qui utilisent le caractère « a ».
- **LIKE 'pa%on' :** ce modèle permet de rechercher les chaines qui commence par « pa » et qui se terminent par « on », comme « pantalon » ou « pardon ».

- **LIKE ‘a_c’ :** peu utilisé, le caractère « _ » (underscore) peut être remplacé par n’importe quel caractère, mais un seul caractère uniquement (alors que le symbole pourcentage « % » peut être remplacé par un nombre incalculable de caractères . Ainsi, ce modèle permet de retourner les lignes « aac », « abc » ou même « azc ».

i. IS NULL / IS NOT NULL

i. DESCRIPTION

Dans le langage SQL, l’opérateur IS permet de filtrer les résultats qui contiennent la valeur NULL. Cet opérateur est indispensable car la valeur NULL est une valeur inconnue et ne peut par conséquent pas être filtrée par les opérateurs de comparaison (cf. égal, inférieur, supérieur ou différent).

ii. SYNTAXE

Pour filtrer les résultats où les champs d’une colonne sont à NULL il convient d’utiliser la syntaxe suivante :

```
SELECT *
FROM `table`
WHERE nom_colonne IS NULL
```

A l’inverse, pour filtrer les résultats et obtenir uniquement les enregistrements qui ne sont pas null, il convient d’utiliser la syntaxe suivante :

```
SELECT *
FROM `table`
WHERE nom_colonne IS NOT NULL
```

A savoir : l’opérateur IS retourne en réalité un booléen, c’est à dire une valeur TRUE si la condition est vrai ou FALSE si la condition n’est pas respectée. Cet opérateur est souvent utilisé avec la condition WHERE mais peut aussi trouver son utilité lorsqu’une sous-requête est utilisée.

iii. EXEMPLE

Imaginons une application qui possède une table contenant les utilisateurs. Cette table possède 2 colonnes pour associer les adresses de livraison et de facturation à un utilisateur (grâce à une clé étrangère). Si cet utilisateur n’a pas d’adresse de facturation ou de livraison, alors le champ reste à NULL.

Table « utilisateur » :

id	nom	date_inscription	fk_adresse_livraison_id	fk_adresse_facturation_id
23	Grégoire	2013-02-12	12	12
24	Sarah	2013-02-17	NULL	NULL
25	Anne	2013-02-21	13	14
26	Frédérique	2013-03-02	NULL	NULL

Il est possible d'obtenir la liste des utilisateurs qui ne possèdent pas d'adresse de livraison en utilisant la requête SQL suivante :

```
SELECT *
FROM `utilisateur`
WHERE `fk_adresse_livraison_id` IS NULL
```

Résultat :

id	nom	date_inscription	fk_adresse_livraison_id	fk_adresse_facturation_id
24	Sarah	2013-02-17	NULL	NULL
26	Frédérique	2013-03-02	NULL	NULL

Les enregistrements retournés montrent bien que seuls les utilisateurs ayant la valeur NULL pour le champ de l'adresse de livraison.

```
SELECT *
FROM `utilisateur`
WHERE `fk_adresse_livraison_id` IS NOT NULL
```

Résultat :

id	nom	date_inscription	fk_adresse_livraison_id	fk_adresse_facturation_id
23	Grégoire	2013-02-12	12	12
25	Anne	2013-02-21	13	14

Les lignes retournés sont exclusivement celles qui n'ont pas une valeur NULL pour le champ de l'adresse de livraison.

j. GROUP BY

i. DESCRIPTION

La commande GROUP BY est utilisée en SQL pour grouper plusieurs résultats et utiliser une fonction de totaux sur un groupe de résultat. Sur une table qui contient toutes les ventes d'un magasin, il est par exemple possible de lister regrouper les ventes par clients identiques et d'obtenir le coût total des achats pour chaque client.

ii. SYNTAXE

De façon générale, la commande GROUP BY s'utilise de la façon suivante :

```
SELECT colonne1, fonction(colonne2)
FROM table
GROUP BY colonne1
```

A noter : cette commande doit toujours s'utiliser après la commande WHERE et avant la commande HAVING.

iii. EXEMPLE

id	client	tarif	date
1	Pierre	102	2012-10-23
2	Simon	47	2012-10-27
3	Marie	18	2012-11-05
4	Marie	20	2012-11-14
5	Pierre	160	2012-12-03

Ce tableau contient une colonne qui sert d'identifiant pour chaque ligne, une autre qui contient le nom du client, le coût de la vente et la date d'achat.

Pour obtenir le coût total de chaque client en regroupant les commandes des mêmes clients, il faut utiliser la requête suivante :

```
SELECT client, SUM(tarif)
FROM achat
GROUP BY client
```

La fonction SUM() permet d'additionner la valeur de chaque tarif pour un même client. Le résultat sera donc le suivant :

client	SUM(tarif)
Pierre	262
Simon	47
Marie	38

La manière simple de comprendre le GROUP BY c'est tout simplement d'assimiler qu'il va éviter de présenter plusieurs fois les mêmes lignes. C'est une méthode pour éviter les doublons.

Juste à titre informatif, voici ce qu'on obtient de la requête sans utiliser GROUP BY.

Requête :

```
SELECT client, SUM(tarif)
FROM achat
```

Résultat :

client	SUM(tarif)
Pierre	262
Simon	47
Marie	38
Marie	38
Pierre	262

Utilisation d'autres fonctions de statistiques

Il existe plusieurs fonctions qui peuvent être utilisées pour manipuler plusieurs enregistrements, il s'agit des fonctions d'agrégations statistiques, les principales sont les suivantes :

- AVG() pour calculer la moyenne d'un set de valeur. Permet de connaître le prix du panier moyen pour de chaque client
- COUNT() pour compter le nombre de lignes concernées. Permet de savoir combien d'achats a été effectué par chaque client
- MAX() pour récupérer la plus haute valeur. Pratique pour savoir l'achat le plus cher
- MIN() pour récupérer la plus petite valeur. Utile par exemple pour connaître la date du premier achat d'un client
- SUM() pour calculer la somme de plusieurs lignes. Permet par exemple de connaître le total de tous les achats d'un client

Ces petites fonctions se révèlent rapidement indispensable pour travailler sur des données.

EXERCICE D'APPLICATION (*2)

Soit le modèle relationnel suivant relatif à la gestion simplifiée des étapes du Tour de France 97, dont une des étapes de

type "contre la montre individuel" se déroula à Saint-Etienne :

— EQUIPE(**CodeEquipe**, NomEquipe, DirecteurSportif)

— COUREUR(**NuméroCoureur**, NomCoureur, #**CodeEquipe**, #**CodePays**)

— PAYS(**CodePays**, NomPays)

— TYPE_ETAPE(**CodeType**, LibelleType)

— ETAPE(**NuméroEtap**, DateEtape, VilleDép, VilleArr, NbKm, #**CodeType)**

— PARTICIPER**(#NuméroCoureur, #NuméroEtape**, TempsRealisé)

— ATTRIBUER_BONIFICATION**(#NuméroEtape, #NuméroCoureur**, km, Rang, NbSecondes)

Exprimez en SQL les requetes suivantes :

1. Quelle est la composition de l'équipe Festina (Numéro, nom et pays des coureurs) ?

2. Quel est le nombre de kilomètres total du Tour de France 97 ?

3. Quel est le nombre de kilomètres total des étapes de type "Haute Montagne" ?

4. Quels sont les noms des coureurs qui n'ont pas obtenu de bonifications ?

5. Quels sont les noms des coureurs qui ont participé à toutes les étapes ?

6. Quel est le classement général des coureurs (nom, code équipe, code pays et temps des coureurs) à l'issue des 13 premières étapes sachant que les bonifications ont été intégrées dans les temps réalisés à chaque étape ?

7. Quel est le classement par équipe à l'issue des 13 premières étapes (nom et temps des équipes) ?

k. HAVING

i. DESCRIPTION

La condition HAVING en SQL est presque similaire à WHERE à la seule différence que HAVING permet de filtrer en utilisant des fonctions telles que SUM(), COUNT(), AVG(), MIN() ou MAX().

ii. SYNTAXE

L'utilisation de HAVING s'utilise de la manière suivante :

```
SELECT colonne1, SUM(colonne2)
FROM nom_table
GROUP BY colonne1
HAVING fonction(colonne2) operateur valeur
```

Cela permet donc de SÉLECTIONNER les colonnes DE la table « nom_table » en GROUPANT les lignes qui ont des valeurs identiques sur la colonne « colonne1" et que la condition de HAVING soit respectée.

Important : HAVING est très souvent utilisé en même temps que GROUP BY bien que ce ne soit pas obligatoire.

id	client	tarif	date_achat
1	Pierre	102	2012-10-23
2	Simon	47	2012-10-27
3	Marie	18	2012-11-05
4	Marie	20	2012-11-14
5	Pierre	160	2012-12-03

Si dans cette table on souhaite récupérer la liste des clients qui ont commandé plus de 40€, toute commandes confondues alors il est possible d'utiliser la requête suivante :

```
SELECT client, SUM(tarif)
FROM achat
GROUP BY client
HAVING SUM(tarif) > 40
```

Résultat :

client	SUM(tarif)
Pierre	162
Simon	47

La cliente « Marie » a cumulée 38 d'achat (un achat de 18 et un autre de 20) ce qui est inférieur à la limite de 40 imposée par HAVING. En conséquent cette ligne n'est pas affichée dans le résultat.

I. ORDER BY

i. DESCRIPTION

La commande ORDER BY permet de trier les lignes dans un résultat d'une requête SQL. Il est possible de trier les données sur une ou plusieurs colonnes, par ordre ascendant ou descendant.

ii. SYNTAXE

Une requête où l'on souhaite filtrer l'ordre des résultats utilise la commande ORDER BY de la sorte :

```
SELECT colonne1, colonne2
FROM table
ORDER BY colonne1
```

Par défaut les résultats sont classés par ordre ascendant, toutefois il est possible d'inverser l'ordre en utilisant le suffixe DESC après le nom de la colonne. Par ailleurs, il est possible de trier sur plusieurs colonnes en les séparant par une virgule. Une requête plus élaborée ressemblerait alors cela :

```
SELECT colonne1, colonne2, colonne3
FROM table
ORDER BY colonne1 DESC, colonne2 ASC
```

A noter : il n'est pas obligé d'utiliser le suffixe « ASC » sachant que les résultats sont toujours classé par ordre ascendant par défaut. Toutefois, c'est plus pratique pour mieux s'y retrouver, surtout si on a oublié l'ordre par défaut.

iii. EXEMPLE

Pour l'ensemble de nos exemples, nous allons prendre un base « utilisateur » de test, qui contient les données suivantes :

id	nom	prenom	date_inscription	tarif_total
1	Durand	Maurice	2012-02-05	145
2	Dupond	Fabrice	2012-02-07	65
3	Durand	Fabienne	2012-02-13	90
4	Dubois	Chloé	2012-02-16	98
5	Dubois	Simon	2012-02-23	27

Pour récupérer la liste de ces utilisateurs par ordre alphabétique du nom de famille, il est possible d'utiliser la requête suivante :

```
SELECT *
FROM utilisateur
ORDER BY nom
```

Résultat :

id	nom	prenom	date_inscription	tarif_total
4	Dubois	Chloé	2012-02-16	98
5	Dubois	Simon	2012-02-23	27
2	Dupond	Fabrice	2012-02-07	65
1	Durand	Maurice	2012-02-05	145
3	Durand	Fabienne	2012-02-13	90

En utilisant deux méthodes de tri, il est possible de retourner les utilisateurs par ordre alphabétique ET pour ceux qui ont le même nom de famille, les trier par ordre décroissant d'inscription. La requête serait alors la suivante :

```
SELECT *
FROM utilisateur
ORDER BY nom, date_inscription DESC
```

Résultat :

id	nom	prenom	date_inscription	tarif_total
5	Dubois	Simon	2012-02-23	27
4	Dubois	Chloé	2012-02-16	98
2	Dupond	Fabrice	2012-02-07	65
3	Durand	Fabienne	2012-02-13	90
1	Durand	Maurice	2012-02-05	145

m. LIMIT

i. DESCRIPTION

La clause LIMIT est à utiliser dans une requête SQL pour spécifier le nombre maximum de résultats que l'ont souhaite obtenir. Cette clause est souvent associé à un OFFSET, c'est-à-dire effectuer un décalage sur le jeu de résultat. Ces 2 clauses permettent par exemple d'effectuer des système de pagination (exemple : récupérer les 10 articles de la page 4).

ii. SYNTAXE

La syntaxe commune aux principaux systèmes de gestion de bases de données est la suivante :

```
SELECT *
FROM table
LIMIT 10
```

Cette requête permet de récupérer seulement les 10 premiers résultats d'une table. Bien entendu, si la table contient moins de 10 résultats, alors la requête retournera toutes les lignes.

n. UNION

i. DESCRIPTION

La commande UNION de SQL permet de mettre bout-à-bout les résultats de plusieurs requêtes utilisant elles-même la commande SELECT. C'est donc une commande qui permet de concaténer les résultats de 2 requêtes ou plus. Pour l'utiliser il est nécessaire que chacune des requêtes à concaténer retournes le même nombre de colonnes, avec les mêmes types de données et dans le même ordre.

A savoir : par défaut, les enregistrements exactement identiques ne seront pas répétés dans les résultats. Pour effectuer une union dans laquelle même les lignes dupliquées sont affichées il faut plutôt utiliser la commande UNION ALL.

ii. SYNTAXE

La syntaxe pour unir les résultats de 2 tableaux sans afficher les doublons est la suivante :

```
SELECT * FROM table1
UNION
SELECT * FROM table2
```

iii. EXEMPLE

Imaginons une entreprise qui possède plusieurs magasins et dans chacun de ces magasins il y a une table qui liste les clients.

La table du magasin n°1 s'appelle « magasin1_client » et contient les données suivantes :

prenom	nom	ville	date_naissance	total_achat
Léon	Dupuis	Paris	1983-03-06	135
Marie	Bernard	Paris	1993-07-03	75
Sophie	Dupond	Marseille	1986-02-22	27
Marcel	Martin	Paris	1976-11-24	39

La table du magasin n°2 s'appelle « magasin2_client » et contient les données suivantes :

prenom	nom	ville	date_naissance	total_achat
Marion	Leroy	Lyon	1982-10-27	285
Paul	Moreau	Lyon	1976-04-19	133
Marie	Bernard	Paris	1993-07-03	75
Marcel	Martin	Paris	1976-11-24	39

Sachant que certains clients sont présents dans les 2 tables, pour éviter de retourner plusieurs fois les mêmes enregistrements, il convient d'utiliser la requête UNION. La requête SQL est alors la suivante :

```
SELECT * FROM magasin1_client
UNION
SELECT * FROM magasin2_client
```

Résultat :

prenom	nom	ville	date_naissance	total_achat
Léon	Dupuis	Paris	1983-03-06	135
Marie	Bernard	Paris	1993-07-03	75
Sophie	Dupond	Marseille	1986-02-22	27
Marcel	Martin	Paris	1976-11-24	39
Marion	Leroy	Lyon	1982-10-27	285
Paul	Moreau	Lyon	1976-04-19	133

Le résultat de cette requête montre bien que les enregistrements des 2 requêtes sont mis bout-à-bout mais sans inclure plusieurs fois les mêmes lignes.

o. UNION ALL

i. DESCRIPTION

La commande UNION ALL de SQL est très similaire à la commande UNION. Elle permet de concaténer les enregistrements de plusieurs requêtes, à la seule différence que cette commande permet d'inclure tous les enregistrements, même les doublons. Ainsi, si un même enregistrement est présent normalement dans les résultats des 2 requêtes concaténées, alors l'union des 2 avec UNION ALL retournera 2 fois ce même résultat.

A savoir : tout comme la commande UNION, il convient que les 2 requêtes retournent exactement le même nombre de colonnes, avec les mêmes types de données et dans le même ordre.

ii. SYNTAXE

La syntaxe de la requête SQL pour unir les résultats des 2 tables est la suivante :

```
SELECT * FROM table1
UNION ALL
SELECT * FROM table2
```

iii. EXEMPLE

La solution a l'exemple précédent sera dans ce cas :

prenom	nom	ville	date_naissance	total_achat
Léon	Dupuis	Paris	1983-03-06	135
Marie	Bernard	Paris	1993-07-03	75
Sophie	Dupond	Marseille	1986-02-22	27
Marcel	Martin	Paris	1976-11-24	39
Marion	Leroy	Lyon	1982-10-27	285
Paul	Moreau	Lyon	1976-04-19	133
Marie	Bernard	Paris	1993-07-03	75
Marcel	Martin	Paris	1976-11-24	39

Le résultat de cette requête montre qu'il y a autant d'enregistrement que dans les 2 tables réunis. A savoir, il y a quelques clients qui étaient présents dans les 2 tables d'origines en conséquent ils sont présents 2 fois dans le résultat de cette requête SQL.

EXERCICE D'APPLICATION (*3)

1- Voici un extrait de la base de données gestion des ventes :

— Produit (**Ref**, Designation, PrixUnitaire, Dimension, **#code_Machine**)

— Vente (**Ncom**, Ref, Qte , DateLiv)

— Commande (**Ncom**, DateCmd, #**CodeClt**,#**Code_Salarie**)

— Produit_concurrent (**Ref**, Designation, PrixUnitaire, PrixUnitaire,Dimension, **#code_Machine,Nom_Concurrent**)

1. Donner la requête qui permet d'obtenir le chiffre d'affaire mensuel de l'année en cours.

2. Donner la requête qui calcule le taux de vente de chaque produit.

3. Donner la requête qui affiche le produit le plus vendu du mois en cour.

4. La table produit concurrent est composée des informations sur les produits vedettes des concurrents ;

Donner la requête qui permet d'ajouter tous les produits du concurrent GleenAlu à la table Produits.

2- Afin d'assurer la qualité des produits attendues par les Clients, l'entreprise cherche à optimiser la gestion des pannes

pouvant survenir dans les infrastructures de production nécessaires à la fabrication du Ciment.

voici un extrait de la base de données :

— TECHNICIEN (**idTech**, nom, prénom, spécialité)

— STATION (**idstat**, nom, Position, coordLat, coordLong,phase)

— MACHINE (**idmach**, état, dateMiseEnService, dateDernièreRévision, #**idStat**)

— TYPEINCIDENT (**id**, description, tempsRéparationPrévu)

— INCIDENT (**idInd**, remarques, dateHeure, dateHeureCloture,#**idmach**,#**idType**)

— INTERVENTION (**idInterv**, dateHeureDébut, dateHeureFin, #**idInd**, #**idTech**)

1. Rédiger la requête SQL permettant d'obtenir la liste par ordre alphabétique des noms et prénoms des techniciens ayant réalisé une intervention sur la Machine identifiée par Ber001.

Ing. KPELOU Manzamasso

2. Rédiger la requête SQL permettant d’obtenir la liste des phases ayant connue un incident de "sur-chauffage" pour le mois Mai 2013.

3. Rédiger la requête SQL permettant d’obtenir le nombre d’incidents non clôturés.

4. Rédiger la requête SQL permettant d’obtenir la liste des noms des stations ayant eu plus de dix incidents.

SQL SOUS-REQUETE

Dans le langage SQL une sous-requête (aussi appelé « requête imbriquée » ou « requête en cascade ») consiste à exécuter une requête à l'intérieur d'une autre requête. Une requête imbriquée est souvent utilisée au sein d'une clause WHERE ou de HAVING pou remplacer une ou plusieurs constantes.

Syntaxe

Il y a plusieurs façons d'utiliser les sous-requêtes. De cette façon il y a plusieurs syntaxes envisageables pour utiliser des requêtes dans des requêtes.

Requête imbriquée qui retourne un seul résultat

L'exemple ci-dessous est un exemple typique d'une sous-requête qui retourne un seul résultat à la requête principale.

```
SELECT *
FROM `table`
WHERE `nom_colonne` = (
    SELECT `valeur`
    FROM `table2`
    LIMIT 1
  )
```

Cet exemple montre une requête interne (celle sur « table2") qui renvoi une seule valeur. La requête externe quant à elle, va chercher les résultats de « table » et filtre les résultats à partir de la valeur retournée par la requête interne.

A noter : il est possible d'utiliser n'importe quel opérateur d'égalité tel que =, >, <, >=, <= ou <>.

Requête imbriquée qui retourne une colonne

Une requête imbriquée peut également retournée une colonne entière. Dès lors, la requête externe peut utiliser la commande IN pour filtrer les lignes qui possèdent une des valeurs retournées par la requête interne. L'exemple ci-dessous met en évidence un tel cas de figure :

```
SELECT *
FROM `table`
WHERE `nom_colonne` IN (
    SELECT `colonne`
    FROM `table2`
    WHERE `cle_etrangere` = 36
  )
```

Requête imbriquée qui retourne un seul résultat

Avec une telle application, il est peut-être utile de connaître la question liée à la dernière réponse ajoutée sur l'application. Cela peut être effectué via la requête SQL suivante :

```
SELECT *
FROM `question`
WHERE q_id = (
    SELECT r_fk_question_id
    FROM `reponse`
    ORDER BY r_date_ajout DESC
    LIMIT 1
  )
```

Requête imbriquée qui retourne une colonne

Imaginons maintenant que l'ont souhaité obtenir les questions liées à toutes les réponses comprises entre 2 dates. Ces questions peuvent être récupérée par la requête SQL suivante :

```
SELECT *
FROM `question`
WHERE q_id IN (
    SELECT r_fk_question_id
    FROM `reponse`
    WHERE r_date_ajout BETWEEN '2013-01-01' AND '2013-12-31'
  )
```

6- SELECT (JOINTURE)

Les jointures en SQL permettent d'associer plusieurs tables dans une même requête. Cela permet d'exploiter la puissance des bases de données relationnelles pour obtenir des résultats qui combinent les données de plusieurs tables de manière efficace.

En général, les jointures consistent à associer des lignes de 2 tables en associant l'égalité des valeurs d'une colonne d'une première table par rapport à la valeur d'une colonne d'une seconde table. Imaginons qu'une base de 2 données possède une table « utilisateur » et une autre table « adresse » qui contient les adresses de ces utilisateurs. Avec une jointure, il est possible d'obtenir les données de l'utilisateur et de son adresse en une seule requête.

On peut aussi imaginer qu'un site web possède une table pour les articles (titre, contenu, date de publication …) et une autre pour les rédacteurs (nom, date d'inscription, date de naissance …). Avec une jointure il est possible d'effectuer une seule recherche pour afficher un article et le nom du rédacteur. Cela évite d'avoir à afficher le nom du rédacteur dans la table « article ».

Il y a d'autres cas de jointures, incluant des jointures sur la même table ou des jointures d'inégalité. Ces cas étant assez particulier et pas si simple à comprendre, ils ne seront pas élaborés sur cette page.

Types de jointures

Il y a plusieurs méthodes pour associer 2 tables ensemble. Voici la liste des différentes techniques qui sont utilisées :

- **INNER JOIN :** jointure interne pour retourner les enregistrements quand la condition est vraie dans les 2 tables. C'est l'une des jointures les plus communes.
- **CROSS JOIN :** jointure croisée permettant de faire le produit cartésien de 2 tables. En d'autres mots, permet de joindre chaque ligne d'une table avec chaque ligne d'une seconde table. Attention, le nombre de résultats est en général très élevé.
- **LEFT JOIN (ou LEFT OUTER JOIN) :** jointure externe pour retourner tous les enregistrements de la table de gauche (LEFT = gauche) même si la condition n'est pas vérifiée dans l'autre table.
- **RIGHT JOIN (ou RIGHT OUTER JOIN) :** jointure externe pour retourner tous les enregistrements de la table de droite (RIGHT = droite) même si la condition n'est pas vérifiée dans l'autre table.
- **FULL JOIN (ou FULL OUTER JOIN) :** jointure externe pour retourner les résultats quand la condition est vraie dans au moins une des 2 tables.
- **SELF JOIN :** permet d'effectuer une jointure d'une table avec elle-même comme si c'était une autre table.
- **NATURAL JOIN :** jointure naturelle entre 2 tables s'il y a au moins une colonne qui porte le même nom entre les 2 tables SQL
- **UNION JOIN:** jointure d'union

a. INNER JOIN

i. DESCRIPTION

Dans le langage SQL la commande INNER JOIN, aussi appelée EQUIJOIN, est un type de jointures très communes pour lier plusieurs tables entre-elles. Cette commande retourne les enregistrements lorsqu'il y a au moins une ligne dans chaque colonne qui correspond à la condition.

ii. SYNTAXE

Pour utiliser ce type de jointure il convient d'utiliser une requête SQL avec cette syntaxe :

```
SELECT *
FROM table1
INNER JOIN table2 ON table1.id = table2.fk_id
```

La syntaxe ci-dessus stipule qu'il faut sélectionner les enregistrements des tables table1 et table2 lorsque les données de la colonne « id » de table1 est égal aux données de la colonne fk_id de table2.

La jointure SQL peut aussi être écrite de la façon suivante :

```
SELECT *
FROM table1
INNER JOIN table2
WHERE table1.id = table2.fk_id
```

La syntaxe avec la condition WHERE est une manière alternative de faire la jointure mais qui possède l'inconvénient d'être moins facile à lire s'il y a déjà plusieurs conditions dans le WHERE.

iii. EXEMPLE

Imaginons une application qui possède une table utilisateur ainsi qu'une table commande qui contient toutes les commandes effectuées par les utilisateurs.

Table utilisateur :

id	prenom	nom	email	ville
1	Aimée	Marechal	aime.marechal@example.com	Paris
2	Esmée	Lefort	esmee.lefort@example.com	Lyon
3	Marine	Prevost	m.prevost@example.com	Lille
4	Luc	Rolland	lucrolland@example.com	Marseille

Table commande :

utilisateur_id	date_achat	num_facture	prix_total
1	2013-01-23	A00103	203.14
1	2013-02-14	A00104	124.00
2	2013-02-17	A00105	149.45
2	2013-02-21	A00106	235.35
5	2013-03-02	A00107	47.58

Pour afficher toutes les commandes associées aux utilisateurs, il est possible d'utiliser la requête suivante :

```
SELECT id, prenom, nom, date_achat, num_facture, prix_total
FROM utilisateur
INNER JOIN commande ON utilisateur.id = commande.utilisateur_id
```

Résultats :

id	prenom	nom	date_achat	num_facture	prix_total
1	Aimée	Marechal	2013-01-23	A00103	203.14
1	Aimée	Marechal	2013-02-14	A00104	124.00
2	Esmée	Lefort	2013-02-17	A00105	149.45
2	Esmée	Lefort	2013-02-21	A00106	235.35

Le résultat de la requête montre parfaite la jointure entre les 2 tables. Les utilisateurs 3 et 4 ne sont pas affichés puisqu'il n'y a pas de commandes associées à ces utilisateurs.

Attention : il est important de noter que si un utilisateur à été supprimé, alors on ne verra pas ses commandes dans la liste puisque INNER JOIN retourne uniquement les résultats ou la condition est vrai dans les 2 tables.

b. LEFT JOIN

i. DESCRIPTION

Dans le langage SQL, la commande LEFT JOIN (aussi appelée LEFT OUTER JOIN) est un type de jointure entre 2 tables. Cela permet de lister tous les résultats de la table de gauche (left = gauche) même s'il n'y a pas de correspondance dans la deuxième table.

ii. SYNTAXE

Pour lister les enregistrements de table1, même s'il n'y a pas de correspondance avec table2, il convient d'effectuer une requête SQL utilisant la syntaxe suivante.

```
SELECT *
FROM table1
LEFT JOIN table2 ON table1.id = table2.fk_id
```

La requête peut aussi s'écrire de la façon suivante :

```
SELECT *
FROM table1
LEFT OUTER JOIN table2 ON table1.id = table2.fk_id
```

Cette requête est particulièrement intéressante pour récupérer les informations de table1 tout en récupérant les données associées, même s'il n'y a pas de correspondance avec table2. A savoir, s'il n'y a pas de correspondance les colonnes de table2 vaudront toutes NULL.

iii. EXEMPLE

Imaginons une application contenant des utilisateurs et des commandes pour chacun de ces utilisateurs. La base de données de cette application contient une table pour les utilisateurs et sauvegarde leurs achats dans une seconde table. Les 2 tables sont reliées grâce à la colonne utilisateur_id de la table des commandes. Cela permet d'associer une commande à un utilisateur.

Table utilisateur :

id	prenom	nom	email	ville
1	Aimée	Marechal	aime.marechal@example.com	Paris
2	Esmée	Lefort	esmee.lefort@example.com	Lyon
3	Marine	Prevost	m.prevost@example.com	Lille
4	Luc	Rolland	lucrolland@example.com	Marseille

Table commande :

utilisateur_id	date_achat	num_facture	prix_total
1	2013-01-23	A00103	203.14
1	2013-02-14	A00104	124.00
2	2013-02-17	A00105	149.45
2	2013-02-21	A00106	235.35
5	2013-03-02	A00107	47.58

Pour lister tous les utilisateurs avec leurs commandes et afficher également les utilisateurs qui n'ont pas effectuées d'achats, il est possible d'utiliser la requête suivante :

```
SELECT *
FROM utilisateur
LEFT JOIN commande ON utilisateur.id = commande.utilisateur_id
```

Résultats :

id	prenom	nom	date_achat	num_facture	prix_total
1	Aimée	Marechal	2013-01-23	A00103	203.14
1	Aimée	Marechal	2013-02-14	A00104	124.00
2	Esmée	Lefort	2013-02-17	A00105	149.45
2	Esmée	Lefort	2013-02-21	A00106	235.35
3	Marine	Prevost	NULL	NULL	NULL
4	Luc	Rolland	NULL	NULL	NULL

Les dernières lignes montrent des utilisateurs qui n'ont effectuée aucune commande. La ligne retourne la valeur NULL pour les colonnes concernant les achats qu'ils n'ont pas effectués.

c. RIGHT JOIN

i. DESCRIPTION

En SQL, la commande RIGHT JOIN (ou RIGHT OUTER JOIN) est un type de jointure entre 2 tables qui permet de retourner tous les enregistrements de la table de droite (right = droite) même s'il n'y a pas de correspondance avec la table de gauche. S'il y a un enregistrement de la table de droite qui ne trouve pas de correspondance dans la table de gauche, alors les colonnes de la table de gauche auront NULL pour valeur.

ii. SYNTAXE

L'utilisation de cette commande SQL s'effectue de la façon suivante :

```
SELECT *
FROM table1
RIGHT JOIN table2 ON table1.id = table2.fk_id
```

La syntaxe de cette requête SQL peut aussi s'écrire de la façon suivante :

```
SELECT *
FROM table1
RIGHT OUTER JOIN table2 ON table1.id = table2.fk_id
```

Cette syntaxe stipule qu'il faut lister toutes les lignes du tableau table2 (tableau de droite) et afficher les données associées du tableau table1 s'il y a une correspondance entre ID de table1 et FK_ID de table2. S'il n'y a pas de correspondance, l'enregistrement de table2 sera affiché et les colonnes de table1 vaudront toutes NULL.

iii. EXEMPLE

Table utilisateur :

id	prenom	nom	email	ville	actif
1	Aimée	Marechal	aime.marechal@example.com	Paris	1
2	Esmée	Lefort	esmee.lefort@example.com	Lyon	0
3	Marine	Prevost	m.prevost@example.com	Lille	1
4	Luc	Rolland	lucrolland@example.com	Marseille	1

Table commande :

utilisateur_id	date_achat	num_facture	prix_total
1	2013-01-23	A00103	203.14
1	2013-02-14	A00104	124.00
2	2013-02-17	A00105	149.45
3	2013-02-21	A00106	235.35
5	2013-03-02	A00107	47.58

Pour afficher toutes les commandes avec le nom de l'utilisateur correspondant il est normalement d'habitude d'utiliser INNER JOIN en SQL. Malheureusement, si l'utilisateur a été supprimé de la table, alors ça ne retourne pas l'achat. L'utilisation de RIGHT JOIN permet de retourner tous les achats et d'afficher le nom de l'utilisateur s'il existe. Pour cela il convient d'utiliser cette requête :

```
SELECT id, prenom, nom, utilisateur_id, date_achat, num_facture
FROM utilisateur
RIGHT JOIN commande ON utilisateur.id = commande.utilisateur_id
```

Résultats :

id	prenom	nom	utilisateur_id	date_achat	num_facture
1	Aimée	Marechal	1	2013-01-23	A00103
1	Aimée	Marechal	1	2013-02-14	A00104

2	Esmée	Lefort	2	2013-02-17	A00105
3	Marine	Prevost	3	2013-02-21	A00106
NULL	NULL	NULL	5	2013-03-02	A00107

Ce résultat montre que la facture A00107 est liée à l'utilisateur numéro 5. Or, cet utilisateur n'existe pas ou n'existe plus. Grâce à RIGHT JOIN, l'achat est tout de même affiché mais les informations liées à l'utilisateur sont remplacées par NULL.

d. FULL JOIN

i. DESCRIPTION

Dans le langage SQL, la commande FULL JOIN (ou FULL OUTER JOIN) permet de faire une jointure entre 2 tables. L'utilisation de cette commande permet de combiner les résultats des 2 tables, les associer entre eux grâce à une condition et remplir avec des valeurs NULL si la condition n'est pas respectée.

ii. SYNTAXE

Pour retourner les enregistrements de table1 et table2, il convient d'utiliser une requête SQL avec une syntaxe telle que celle-ci :

```
SELECT *
FROM table1
FULL JOIN table2 ON table1.id = table2.fk_id
```

Cette requête peut aussi être conçu de cette façon :

```
SELECT *
FROM table1
FULL OUTER JOIN table2 ON table1.id = table2.fk_id
```

La condition présentée ici consiste à lier les tables sur un identifiant, mais la condition peut être définie sur d'autres champs.

iii. EXEMPLE

Prenons l'exemple d'une base de données qui contient une table utilisateur ainsi qu'une table commande qui contient toutes les ventes.

Table utilisateur :

id	prenom	nom	email	ville	actif
1	Aimée	Marechal	aime.marechal@example.com	Paris	1
2	Esmée	Lefort	esmee.lefort@example.com	Lyon	0
3	Marine	Prevost	m.prevost@example.com	Lille	1
4	Luc	Rolland	lucrolland@example.com	Marseille	1

Table commande :

utilisateur_id	date_achat	num_facture	prix_total
1	2013-01-23	A00103	203.14
1	2013-02-14	A00104	124.00
2	2013-02-17	A00105	149.45
3	2013-02-21	A00106	235.35
5	2013-03-02	A00107	47.58

Il est possible d'utiliser FULL JOIN pour lister tous les utilisateurs ayant effectué ou non une vente, et de lister toutes les ventes qui sont associées ou non à un utilisateur. La requête SQL est la suivante :

```
SELECT id, prenom, nom, utilisateur_id, date_achat, num_facture
FROM utilisateur
FULL JOIN commande ON utilisateur.id = commande.utilisateur_id
```

Résultat :

id	prenom	nom	utilisateur_id	date_achat	num_facture
1	Aimée	Marechal	1	2013-01-23	A00103
1	Aimée	Marechal	1	2013-02-14	A00104
2	Esmée	Lefort	2	2013-02-17	A00105
3	Marine	Prevost	3	2013-02-21	A00106
4	Luc	Rolland	NULL	NULL	NULL
NULL	NULL		5	2013-03-02	A00107

Ce résultat affiche bien l'utilisateur numéro 4 qui n'a effectué aucun achat. Le résultat retourne également la facture A00107 qui est associée à un utilisateur qui n'existe pas (ou qui n'existe plus). Dans les cas où il n'y a pas de correspondance avec l'autre table, les valeurs des colonnes valent NULL.

EXERCICE D'APPLICATION(*4)

1- Soit la base de données suivante :

Etudiant

numéro_carte_etudiant	Nom	Prénom	Date_naissance	Section
01234567	Ben Salah	Ahmed	12/08/1988	Informatique
01234568	Ben Mahmoud	Sami	02/09/1990	Math
01234569	Marzougui	Rami	23/01/1988	Informatique

Matière

code_matière	nom_matière	coefficient
12508	Base de données	1.5
12518	Algorithme	3

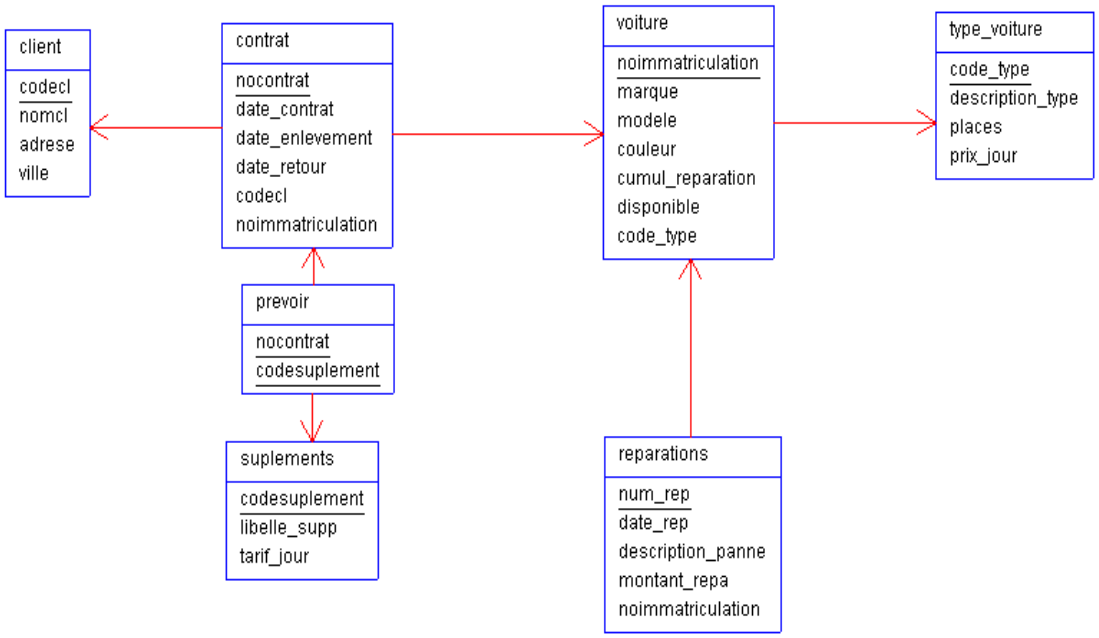
Note

numéro_carte_etudiant	code_matière	note_examen
01234567	12508	15.5
01234567	12518	5.5
01234568	12518	10.5
01234569	12518	8.75

Ecrire les commandes SQL permettant de rechercher :

1. La liste de tous les étudiants.
2. Nom et coefficient des matières.
3. Les numéros des cartes d'identité des étudiants dont la moyenne entre 7 et 12.
4. La liste des étudiants dont le nom commence par 'ben'.
5. Le nombre des étudiants qui ont comme matière '12518'.
6. La somme des coefficients des matières.
7. Les noms des étudiants qui une note_examen >10.
8. Afficher les noms et les coefficients des matières étudier par l'étudiant "01234568".

2- Dans le cadre de son développement l'Agence **TOP LOCATION** décide d'informatiser son système d'information. Après analyse du système on vous a proposé le **MLD** suivant :



1. Créer la base de données "**Location**" avec les différentes tables et relations.
2. Réaliser les requêtes suivantes :

a) Afficher la liste des clients triée par Nom ;

b) Afficher la liste des contrats, triée par Date contrat ;

c) Afficher la liste des contrats d'un client donné ;

d) Afficher la liste des voitures disponibles ;

e) Afficher la liste des réparations pour une voiture donnée ;

f) Afficher la réparation la plus chère;

g) Afficher le nombre de voiture par type ;

h) Afficher le nombre de réparations par voiture ;

i) Afficher le nombre de réparations par type_voiture ;

j) Afficher le montant des contrats par client ;

k) Afficher le montant d'un contrat donné ;

l) Afficher les voitures ayant au moins une date de contrat identique à eux dela voiture immatriculée 3567-ww.

3. Soit le modèle relationnel suivant relatif à la gestion des notes annuelles d'une promotion d'étudiants :

— ETUDIANT(**NEtudiant**, Nom, Prénom)

— MATIERE(**CodeMat**, LibelléMat, CoeffMat)

— EVALUER**(#NEtudiant, #CodeMat**, Date, Note)

Remarque : les clés primaires sont soulignées et les clés étrangères sont marquées par #

Exprimez en SQL les requêtes suivantes :

1. Quel est le nombre total d'étudiants ?

2. Quelles sont, parmi l'ensemble des notes, la note la plus haute et la note la plus basse ?

3. Quelles sont les moyennes de chaque étudiant dans chacune des matières ?

4. Quelles sont les moyennes par matière ? Avec la vue MGETU de la question 3 (MOYETUMAT)

5. Quelle est la moyenne générale de chaque étudiant ? Avec la vue MGETU de la question 3 (MOYETUMAT)

6. Quelle est la moyenne générale de la promotion ? Avec la vue MGETU de la question 5

7. Quels sont les étudiants qui ont une moyenne générale supérieure ou égale à la moyenne générale de la promotion ? Avec la vue MGETU de la question 5

4. Soit la base de données intitulée "gestion_projet" permettant de gérer les projets relatifs au développement de logiciels.

Elle est décrite par la représentation textuelle simplifiée suivante :

— Developpeur (**NumDev**, NomDev, AdrDev, EmailDev, TelDev)

— Projet (**NumProj**, TitreProj, DateDeb, DateFin)

— Logiciel (**CodLog**, NomLog, PrixLog, #**NumProj**)

— Realisation (**#NumProj, #NumDev**)

Ecrire les requêtes SQL permettant :

1. D'afficher les noms et les prix des logiciels appartenant au projet ayant comme titre « gestion de stock », triés dans l'ordre décroissant des prix .

2. D'afficher le total des prix des logiciels du projet numéro 10. Lors de l'affichage, le titre de la colonne sera « cours total du projet ».

3. Afficher le nombre de développeurs qui ont participé au projet intitulé « gestion de stock »

4. Afficher les projets qui ont plus que 5 logiciels

5. Les numéros et noms des développeurs qui ont participés dans tous les projets.

6. Les numéros de projets dans lesquelles tous les développeurs y participent dans sa réalisation.

5. Ci-après, on donne la représentation textuelle simplifiée d'une base de données concernant un cycle de formation destiné

à des étudiants. Il regroupe un ensemble de matières. On considère que chaque enseignant n'enseigne qu'une seule matière et

qu'à la fin du cycle de formation, une note par matière, est attribuée à chaque étudiant. D'autre par, les étudiants peuvent

ne pas suivre les mêmes matières.

— ETUDIANT(**CodeEt**, NomEt, DatnEt)

— MATIERE(**CodeMat**, NomMat, CoefMat)

— ENSEIGNANT(**CodeEns**, NomEns, GradeEns, #**CodeMat**)

— NOTE(**#CodeEt, #CodeMat**, note)

Ecrire les requêtes SQL permettant d'afficher :

1. Les informations relatives aux étudiants (Code, Nom et Date de naissance) selon l'ordre alphabétique croisant du nom

2. Les noms et les grades des enseignants de la matière dont le nom est 'BD'.

3. La liste distincte formée des noms et les coefficients des différentes matières qui sont enseignées par des enseignants de grade 'Grd3'.

4. La liste des matières (Nom et Coefficient) qui sont suivies par l'étudiant de code 'Et321'.

5. Le nombre d'enseignants de la matière dont le nom est 'Informatique'